



 Latest updates: <https://dl.acm.org/doi/10.1145/3704436>

SURVEY

Democratizing Container Live Migration for Enhanced Future Networks - A Survey

WISSEM SOUSSI, ZHAW Zurich University of Applied Sciences,
Winterthur, ZH, Switzerland

GÜRKAN GÜR, ZHAW Zurich University of Applied Sciences,
Winterthur, ZH, Switzerland

BURKHARD STILLER, University of Zurich, Zurich, ZH, Switzerland

Open Access Support provided by:

ZHAW Zurich University of Applied Sciences

University of Zurich



PDF Download
3704436.pdf
05 March 2026
Total Citations: 8
Total Downloads:
2913

Published: 23 December 2024

Online AM: 14 November
2024

Accepted: 04 November 2024

Revised: 29 October 2024

Received: 26 April 2024

Citation in BibTeX format

Democratizing Container Live Migration for Enhanced Future Networks - A Survey

WISSEM SOUSSI, Institute of Computer Science (InIT), Zurich University of Applied Sciences ZHAW, Winterthur, Switzerland and Department of Informatics (IfI), University of Zurich, Zurich, Switzerland

GÜRKAN GÜR, Institute of Computer Science (InIT), Zurich University of Applied Sciences ZHAW, Winterthur, Switzerland

BURKHARD STILLER, Institut für Informatik (IfI), Communication Systems Group, University of Zurich Faculty of Economics Business Administration and Information Technology, Zurich, Switzerland

Emerging cloud-centric networks span from edge clouds to large-scale datacenters with shared infrastructure among multiple tenants and applications with high availability, isolation, fault tolerance, security, and energy efficiency demands. Live migration (LiMi) plays an increasingly critical role in these environments by enabling seamless application mobility covering the edge-to-cloud continuum and maintaining these requirements. This paper presents a comprehensive survey of recent advancements that democratize LiMi, making it more applicable to a broader range of scenarios and network environments both for virtual machines (VMs) and containers, and analyzes LiMi's technical underpinnings and optimization techniques. It also delves into the issue of connections handover, presenting a taxonomy to categorize methods of traffic redirection synthesized from the existing literature. Finally, it identifies technical challenges and paves the way for future research directions in this key technology.

CCS Concepts: • **Security and privacy** → **Network security**; • **Networks** → **Cloud computing**; *Network management*; **Network dynamics**; • **Computer systems organization** → **Dependable and fault-tolerant systems and networks**;

Additional Key Words and Phrases: Service live migration, traffic session handover, edge-to-cloud continuum, Moving Target Defense (MTD)

ACM Reference Format:

Wissem Soussi, Gürkan Gür, and Burkhard Stiller. 2024. Democratizing Container Live Migration for Enhanced Future Networks - A Survey. *ACM Comput. Surv.* 57, 4, Article 97 (December 2024), 37 pages. <https://doi.org/10.1145/3704436>

This work was supported partially by (a) the University of Zürich UZH, Switzerland and (b) the Horizon Europe Framework Program's project NATWORK, Grant Agreement No. 101139285 funded by the Swiss State Secretariat for Education, Research, and Innovation SERI, under Contract No. 22.00642.

Authors' Contact Information: Wissem Soussi, Institute of Computer Science (InIT), Zurich University of Applied Sciences ZHAW, Winterthur, Zurich, Switzerland and Department of Informatics (IfI), University of Zurich, Zurich, Zurich, Switzerland; e-mail: sous@zhaw.ch; Gürkan Gür, Institute of Computer Science (InIT), Zurich University of Applied Sciences ZHAW, Winterthur, Switzerland; e-mail: gueu@zhaw.ch; Burkhard Stiller, Institut für Informatik (IfI), Communication Systems Group, University of Zurich Faculty of Economics Business Administration and Information Technology, Zurich, Zurich, Switzerland; e-mail: stiller@tik.ee.ethz.ch.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 0360-0300/2024/12-ART97

<https://doi.org/10.1145/3704436>

1 Introduction

Future networks, whether traditional enterprise networks, **wide area networks (WAN)**, or mobile telecommunication networks (e.g., 6G or NextG systems), are going to be cloud-centric, inheriting the characteristics of cloud infrastructures. This transformation leads to prevalent traits ranging from the virtualization of resources to programmable networks, both used to facilitate the control of network functions managing the infrastructure and the connected applications provided to end-users. **Software-defined networking (SDN)** [1], **Programming Protocol-independent Packet Processors (P4)** [2], and **Network Function Virtualization (NFV)** [3] enable automated and cognitive orchestration required in distributed and heterogeneous future networks, spanning from limited edge nodes close to the end-users to distant large-scale datacenters and everything in between. This brings the focus on the concept of edge-to-cloud continuum [4], where the system's complexity grows with dependent intermediaries, and the availability, isolation, fault tolerance, and security of applications become more challenging to preserve.

The **Live Migration (LiMi)**, or hot migration, of virtualized services, whether hosted as **Virtual Machines (VM)** or container runtimes, consists of migrating a service's instance from the host machine to a new one while the service remains available to the end-users. This opposes cold migration, where a migrated service needs to be stopped first, hindering the continuity of its functions. This disruption, in the order of seconds, can be undesirable when dealing with live or critical applications such as live video streams and gaming, autonomous vehicles, healthcare systems, and industrial control systems. LiMi improves the isolation of services in multi-tenant hosting infrastructures, improves the adaptability to new regulations and security policies (whether for technical or geopolitical considerations), enhances networks' resilience against cyber threats, and maintains fault tolerance and availability of networked services. It is also used for other relevant goals, such as managing and optimizing the carbon footprint associated with networked applications. This aligns with the imperative of developing networking practices that comply with the Net Zero emissions and climate-neutrality targets set or considered by over 100 countries in UN organizations and various international agreements, including the Paris Agreement [5]. Broadly, LiMi plays a crucial role in today's network evolution to ensure and improve the previously mentioned requirements of availability, isolation, fault tolerance, security, and resource efficiency of applications in the edge-to-cloud continuum.

This survey aims at comprehensively examining and analyzing the technological landscape and algorithmic advances associated with LiMi of VMs and containers within contemporary and future network environments. To the best of the authors' knowledge, this work presents the first survey that covers a literature study of LiMi from the perspective of enabling this technology seamlessly, i.e., as an innate capability, for all relevant networking and cloud computing applications – ranging from services using GPU cores for AI applications to services using **Trusted Execution Environments (TEE)** to guarantee zero-trust security properties in multi-tenant cloud environments. Through this comprehensive study, it aims at contributing valuable insights for researchers, practitioners, and decision-makers involved in cloud computing, virtualization, and related domains.

1.1 Related Surveys

Previous surveys on the LiMi topic have been done until recently. A comparison of the coverage provided by these surveys with that of the current survey is summarized and presented in Table 1.

El-Said et al. [6] presented a survey on different cost models of the **pre-copy** LiMi. This optimization method depended on specific parameters to be fixed, detailed in Section 3. While this work concerned methods that can also be used in containerized and microservices environments, it only focused on the literature applying them in VM-based environments such as VMware vSphere, Microsoft Hyper-V, KVM, and Xen hypervisors. This and other surveys [7–10] indicate the maturity

Table 1. Comparison of the Key Contributions of This Article's Survey Compared to Other Existing Surveys on LiMi

Criteria/Surveys	[6–10]	[11, 12]	[13]	[14]	[15]	[16]	This article
VM LiMi	✓	✓	✓	✓	∂	✓	✓
Container LiMi	✗	∂	✓	✓	✓	✓	✓
Migration optimization	✓	✓	✓	✓	✓	✓	✓
Container democratization	✗	✗	✗	✗	✗	∂	✓
Traffic relocation	✗	✗	✗	∂	∂	✗	✓
Cloud	✓	✓	✗	✗	✗	✓	✓
MEC	✗	✓	✓	✓	✗	✓	✓
Fog Computing	∂	✗	✓	✓	✓	✓	✓
Micro services	✗	✗	✗	✗	✗	✓	✓
Migration for security	✗	✗	✗	✗	✗	∂	✓
Stateful vs stateless	✗	✗	✗	✓	✗	✗	✓

✓ designates the coverage of the respective topic, ∂ designates partial coverage of the respective topic ✗ designates no coverage of the respective topic.

of VM-based LiMi technology and the logical transfer of the same methods to container-based LiMi.

Wang et al. [11] made a distinction between LiMi in clouds and **multi-access edge computing (MEC)** environments, considering the different objectives in both cases and the limited resources in edge nodes when dealing with a MEC setup. A focus was then given to the migration strategy based on users' movements and its modeling to decide when and where to migrate a MEC application across different **radio access networks (RANs)**.

Tianzhang et al. [13] extensively covered the taxonomy and types of LiMi in the cloud for both VMs and containers. Their work also focused on optimization methods and their tradeoffs regarding migration time, service downtime, data transfer size, stability, and complexity.

Kaur et al. [16] presented recent innovative LiMi works on containerized applications, displaying the contrast with VM-based LiMi and covering the usage of it in fog, MEC, cloud, and microservices environments, with a particular focus on migrating VNF-based network services, relevant to contemporary telecommunication networks. The survey did not present the difference in security between the stateless service LiMi and the stateful one. It omitted the traffic relocation challenge, which heavily affects the availability of the service and differs in technicality when live migrating within one cluster, in contrast to a multi-cluster geo-distributed network.

Overall, there is a general lack of surveys that systematically address the intricacies and challenges associated with the task of traffic relocation, which can differ from methods on layer four protocols (e.g., TCP and UDP), to layer seven protocols (e.g., HTTP), as well as the nature of communication scenarios. These scenarios encompass instances of short-lived connections, characterized by the frequent re-establishment of end-to-end sessions, in contrast to long-lived connections where the end-to-end session persists over extended durations. Section 4 is dedicated to exploring this critical area of study, further expanding the scope of coverage, which includes all major aspects as outlined in Table 1.

1.2 Selection Method

This survey study was conducted based on the established search of predefined themes relevant to LiMi. As the existing LiMi literature is extensive and rich, this article's approach was first to determine new contributions with this survey focusing on two major themes that are not yet

Table 2. Section-wise Research Themes and Search Keywords

Section	Theme	Search Keywords	No. of Articles
Section 3	Background	<i>live migration</i>	12
Section 3.3	LiMi performance optimization	<i>live migration, pre-copy, post-copy</i>	9
Section 4	LiMi traffic relocation methods	<i>live migration, connection handover, traffic relocation/redirection</i>	11
Section 5.1	Container LiMi’s democratization	<i>live migration, container, TEE, GPU, 5G</i>	11
Section 5.2	LiMi in microservices	<i>live migration, microservices, Kubernetes, Docker</i>	4
Section 5.3	LiMi in telecommunication networks (MEC and Fog)	<i>live migration, migration in the edge, service placement</i>	14
Section 5.4.1	LiMi’s security	<i>live migration, secure live migration</i>	6
Section 5.4.2	LiMi for security	<i>live migration, moving target defense</i>	6

extensively covered: (1) the task of traffic relocation and connection handover necessary in LiMi, and (2) new work extending the usage of LiMi in new critical scenarios. The search process has been conducted in two phases:

- **Phase 1:** Based on existing survey articles collected, amounting to 11 as listed in Table 1, the collection of recent and relevant LiMi work was performed. As noted above, this activity also served us to determine which topics were covered extensively and which were not in these surveys, and to differentiate our survey from them. As the final output, this phase led to the topic-based aggregation of these formerly surveyed articles as the baseline set of approaches.
- **Phase 2:** This had been extended by conducting our own literature search exploring various digital libraries, such as ACM and IEEE, using their respective search engines as well as Google Scholar. Finally, the augmented body of work in this survey here is presented in a structural manner (i.e., with appropriate groupings and corresponding sections), also synthesizing technical challenges and future research directions based on the analysis in the following parts. Table 2 presents the section-wise LiMi themes covered in this survey with the search keywords used and the number of articles presented per theme. Please note, the total number of articles in this table is higher than the final number of presented articles, since some could be included in multiple themes.

Tables 6–10 enlist the literature reviewed in the corresponding sections for their respective themes. The references extensively studied in this survey amount to 55 from 121 references published between 1982 and 2024, with circa 60% of references published since 2018 and 90% since 2012, indicating a proper coverage of recent works.

1.3 Survey Structure

The survey structure is depicted in Figure 1 and incorporates the following sections.

Section 2 describes the relevance of LiMi and details the critical features that the technology would enable. Section 3 presents the background of LiMi, how it is performed, and its evolution from a technical perspective, moving from VMs to containers, as well as the contrast between stateful and stateless migrations. This section also dives deeper into the optimization methods for stateful LiMi to reduce the migration time and especially the downtime of moved applications, then covers the technical implementation of such methods with the **Checkpoint/Restore In Userspace (CRIU)** library [17], a de-facto standard for containers LiMi. Section 4 shifts the

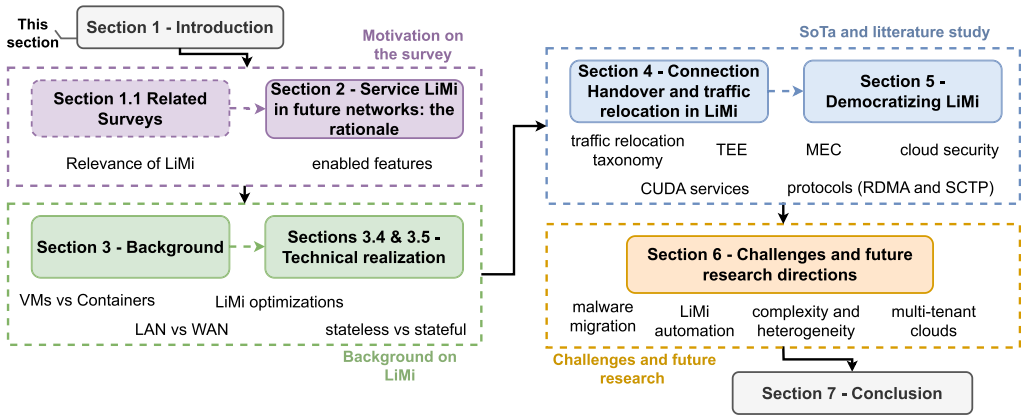


Fig. 1. The structure of this survey and the covered topics.

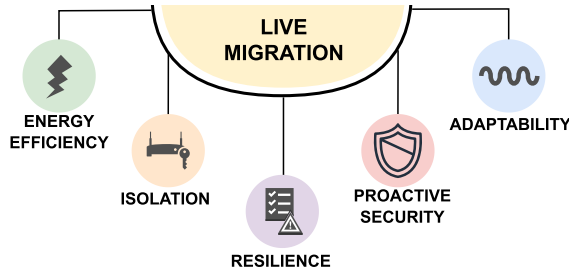


Fig. 2. Essential features in ICT systems enhanced by LiMi.

focus on the methods of traffic redirection for LiMi and the challenges faced to obtain seamless session-based, long-lived connection handovers. Section 5 presents the **state-of-the-art (SOTA)** on LiMi, the technical novelty for its democratization to include GPU-dependent AI applications (using CUDA cores [18]), high-performance network interfaces and protocols (for telecommunication core network functions), and applications running on **TEE** [19]. This section also covers its application in cloud computing, fog computing, and MEC [20]. Here, additional focus is given to the security of LiMi and to LiMi for the protection of networked services and infrastructures. Section 6 elaborates on the challenges yet to be solved for broader and conventional adoption of container LiMi in corner use cases and scenarios where service requirements and infrastructure setups are stringent, proposing as well possible solutions to investigate. Finally, Section 7 discusses future research directions.

2 Service LiMi in Future Networks: The Rationale

Service LiMi is pivotal in many properties and features required in **information and communication technology (ICT)** systems, as depicted in Figure 2.

2.1 Improved Isolation in Multi-tenant Infrastructures (Non-static Hosting)

Most of the services hosted in the cloud share the infrastructure with different tenants and owners of other services. The hypervisors managing the service instances are often vulnerable, and while vulnerabilities that allow direct unauthorized access to third-party resources are very few and quick to be disclosed and patched, other more passive attacks, like side-channel attacks, are

more common. Side channel attacks allow attackers to get sensitive data on neighboring VMs or containers hosted within the same node [21–23]. By dynamically migrating such VMs and containers, the time exploitation window of such attacks is sensitively reduced as services move in different nodes and locations of the cloud infrastructure. Recent studies and solutions also deal with the isolation problem using the TEE technology. In this scenario, containers and VMs run in encrypted enclaves guaranteeing confidentiality even in a multi-tenant system. Enabling LiMi for such TEE-protected systems becomes relevant to provide an additional layer of isolation while removing static-hosting constraints on services (further discussed in Section 5.1.6).

Another feature enabled by LiMi against multi-tenant threats is the on-demand allocation of a private physical node within the cluster that would only be used by one applicant. This entails the LiMi of the applicant's services to the new node and the displacement of unrelated resources to other nodes. LiMi to a dedicated physical node is also critical in scaling out the service and in **High-Performance Computing (HPC)** to avoid performance interference and ensure consistency throughout the service execution.

2.2 Adaptability to Changing Policies and Regulations

An important property needed for cloud-based infrastructures is the possibility to dynamically adapt to changing policies and regulations. For instance, with the advent of the **European General Data Protection Regulation (EU GDPR)**, the personal data that is generated in Europe cannot be stored or processed outside of the European territory unless an “Adequacy Decision” is offered by the European Commission or a user explicitly consents (and can revoke) the transfer of his data [24]. At the occurrence of this kind of new regulations, especially when frequently happening (i.e., due to shifting geopolitical conditions), LiMi would allow to swiftly adapt with minimal efforts by migrating the resources, data, and processing applications to the required location (assuming the infrastructure at such location is available).

2.3 Improved Proactive and Reactive Security

As seen with side-channel attacks, LiMi can be used as an additional layer of security for large ICT systems, as the increasing size and complexity also lead to a higher attack surface that attackers can explore and exploit. In this case, LiMi can be used to perform **Moving Target Defense (MTD)** operations. MTD is defined by the **National Institute of Standards and Technology (NIST)** as “*the concept of controlling change across multiple system dimensions in order to increase uncertainty and apparent complexity for attackers, reduce their window of opportunity, and increase the costs of their probing and attack efforts.*” [25]. MTD's objective is to reduce the asymmetry that usually exists between attackers and defenders, as on one hand attackers from the internet have the opportunity to probe the target and analyze its weaknesses to be exploited, while on the other hand, defenders have no information on the attacker and its capabilities but can only guess. With MTD operations, the data that attackers gather becomes obsolete and unusable for the attacker. In this sense, the LiMi of services in a cloud-based infrastructure would allow avoiding various threats proactively, as a means of prevention, and reactively, to mitigate and evade an occurring attack. Further details on the security use case are provided in Section 5.4.

2.4 Increased Availability and Resilience

These fundamental and classic requirements have always been targeted at the different levels of ICT systems: software engineering, hardware, infrastructure, and networking. The different levels have possibly met and culminated in the virtualization technology, followed by the microservices' paradigm [26], which is changing the way software is developed, served, and managed in new cloud-based networking infrastructures. It is in such an environment that LiMi sees its best

prospect and usage, as it uses the characteristics of microservices-based ICT systems: modularity, portability, replicability, distribution, and autonomous orchestration. In microservices, availability and fault tolerance are mainly improved with the replication of the applications and the load balancing of requests to the various replicas. LiMi can maintain the same level of availability across different distributed nodes while containing the number of replicas, which consume more resources. Moreover, traditional microservice replication only applies to stateless microservices, i.e., applications that only need the VM/container image to start serving the service, as it does not make use of dynamic contextual data or state. However, with LiMi, both stateless and stateful running applications can be migrated from one node to another, making it possible also to migrate the clients' connections. In MEC and Fog computing scenarios, this also allows bringing the service closer to moving users, improving the availability and **quality of service (QoS)** in use cases such as smart connected vehicles' and mobile devices' use cases [11, 27–29].

2.5 Energy and Carbon Footprint Control

The power consumption of datacenters and cloud infrastructures in 2022 was 240–340 TWh, about 1%–1.3% of global electricity use [30]. Despite notable improvements in efficiency, the increase of workloads managed by data centers has led to a significant rise in energy consumption, experiencing an annual growth rate ranging between 20%–40% [30].

LiMi serves as a mechanism for regulating and overseeing energy utilization. Restricting application replicas while maintaining high service availability via migration offers a means to limit resource and energy consumption, as previously discussed. An alternative approach, referred to as “consolidation” involves live migrating all active applications within a cluster to a minimal number of server nodes, as demonstrated in Hermenier et al.'s work on Xen VM migrations [31]. This practice enables the hibernation of idle server nodes during periods of decreased resource demand, effectively mitigating energy consumption. In fact, even at a low 10% CPU utilization, the power consumption remains high, exceeding 50% of the host's maximal power consumption [32]. Conversely, in certain higher workloads, distributing applications across more nodes can result in reduced waste heat dissipation, thereby lowering the energy consumed by cooling systems, along with improving the user's QoS. Huang et al. [33] present a consolidation strategy that considers both the QoS and power consumption of VM LiMi, demonstrating that the power overhead of migration is negligible compared to the resulting savings from consolidation.

Beyond the energy reduction goal, achieving carbon neutrality involves a focus on using renewable energy. Rozite et al. [30] present hyperscale datacenter operators as the biggest purchasers of corporate renewable energy procurement through **power purchase agreements (PPAs)**. In 2021, Google, Meta, Apple, and Microsoft generated or purchased renewable energy to match 100% of their consumption (although they keep using non-renewable power to cover their energy overloads) [30]. Amazon, the biggest consumer of renewable energy by twofold compared to the other providers, covers 85% of its consumption. In this context, the LiMi of resources to destination clusters operating on green energy emerges as a viable strategy in a multi-tenant environment. This approach could serve as a more granular complementary solution to traditional PPAs, offering instances' owners the means to mitigate their carbon footprint in accordance with their individual green policies.

3 Background

LiMi and most of its optimization techniques originated with VMs [34–41], but the same principles and optimizations were transferred to container runtimes as well. LiMi can be classified as *inter-cluster* or *multi-cluster*. In *inter-cluster* migrations the application is moved between nodes of the same cluster, and this can be performed in two different ways:

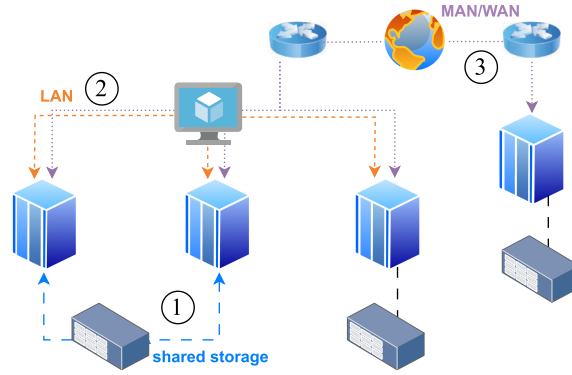


Fig. 3. Types of LiMi in a multi-cloud environment.

- *Over shared storage*, where the volume and data of the application are immovable and placed in a storage network, which will be mounted on the application instance in the new node (see Marker ① in Figure 3) [37].
- *Over the network*, where the volume and data of the application are attached to the running instance and are moved all together through the network (see Marker ② in Figure 3) [34].

In *multi-cluster* migrations, the application is transferred from one cluster to the other, and this can be extended to multi-cloud and/or multivendor migrations [42]. As the destination in these scenarios is far from the source, the storage network cannot be the same as it will create a significant communication overhead in latency and traffic, hindering both application QoS and network resource consumption. The complete application transfer then happens over the WAN/internet, as shown by Marker ③ of Figure 3, making it a slower process compared to inter-cluster migrations, depending on the geographical distance between source and destination nodes, the quality of the physical connection, and the size of the application to migrate [39].

3.1 VM vs. Container LiMi

This subsection points out key differences between VM LiMi and container LiMi, summarized in Table 3.

Concerning literature coverage, scientific contributions, and technological maturity, VM LiMi is advantageous as it came first and has been and still is extensively used in critical infrastructures by industries and enterprises.

In theory, VMs are also more portable than containers, as containers are dependent on libraries of the host's OS, thus requiring the same libraries (or OS) on both source and destination hosts. However, in real-life LiMi scenarios, this difference between VMs and containers in terms of portability/cross-migration could be reversed. This is because VM LiMi implementations, especially with live optimizations, are mostly licensed or OS dependent (e.g., *OpenVZ*). However, almost all container's LiMIs, especially with live optimizations, currently use the open-source library CRIU (further described in Section 3.4), which can run on any recent Linux kernel, making it more usable across a variety of distributions and infrastructures. This process of "democratization" of container LiMi is further discussed in Section 5.1.

Because VMs are much bigger in size, they take longer to migrate, and for the time constraint requirements of LiMi, this is a significant drawback. On the other hand, container images are much smaller and lightweight, leading to faster LiMi with reduced data transfer and network overhead. Kotikalapudi et al. [43] demonstrated that migrating the identical service with a 66% workload,

Table 3. VM vs. Container LiMi

Factors	VM migration	Container migration
Maturity	Mature enterprise use of migration technology.	Newer and less documented in enterprise use cases.
Flexibility	VMs are more portable as they are independent of the host OS, enabling, in theory, LiMi with fewer/no constraints on the host machine.	Containers depend on the host's OS libraries, thus requiring the same libraries or OS on the destination host.
	VM LiMi's optimizations mostly depend on specific implementations, thus requiring the same hypervisor at the destination.	Almost all container LiMIs are currently accomplished with CRIO, which runs on any Linux OS, independent of the container orchestrator.
Efficiency	VMs are generally large implementations of an entire ecosystem with multiple software/applications $\Rightarrow$ reduced control of migration granularity.	Containers are small single applications running as modular components in a microservices environment $\Rightarrow$ high control of migration granularity.
	A VM image size is in the order of gigabytes, requiring greater bandwidth consumption during migration.	Container images are in the order of megabytes, making the migration less taxing on the network.
Performance	The significant data transfer causes a longer migration downtime (during which service is unavailable).	Lightweight container images cause shorter migration downtime.
	VM warm startup (which excludes the first boot) is in the order of seconds. The startup time contributes to increasing the migration downtime.	Container warm startup is in the order of milliseconds. However, the first boot (cold startup) is similar to or longer than the VM's boot due to <i>filesystem remap</i> .

encapsulated within a system container, exhibits a 45% acceleration and a 75% reduction in downtime compared to its encapsulation in a VM. For a meaningful comparison, the authors used **Linux containers (LXC)**, which are full system containers providing an entire Linux OS with its kernel, as opposed to application containers such as Docker, which only package the application and its dependencies (further reducing the container's size). Kotikalapudi et al. also used the same migration method on both virtualization technologies, i.e., a pre-copy optimization (further described in Section 3.3.1).

Because VMs have to initiate a complete **operating system's (OS)** kernel, VM-based applications have significantly longer spin-up times [16], meaning it takes longer for a service to be available and served at the instantiation phase (in the order of minutes for VMs and few seconds, often milliseconds, with containers). These times can be added to the downtime of the service during the migration (i.e., the time where the service is not available to the end-users). This is relevant to stateful applications, as further clarified in the following Section (3.2).

Another important difference is the way applications are implemented when using VMs compared to containers. Using VMs, software developers tend to implement a monolithic application with all the components needed in one single VM. On the other hand, when using containers developers follow the microservices approach, implementing multiple smaller services in portable and relatively independent containers. Hence the relevance and importance of LiMi research in microservices architecture for future networks is evident, as described in the previous section.

3.2 Stateless LiMi

LiMi methods and algorithms also differ based on whether the service to migrate is stateless or stateful. Stateless services are, for instance, the majority of firewalls, DNS servers, or network gateways that do not have a dynamic context to keep and only have rare changes in their configuration that could be added preemptively to the system image used to start the service. In this case, the

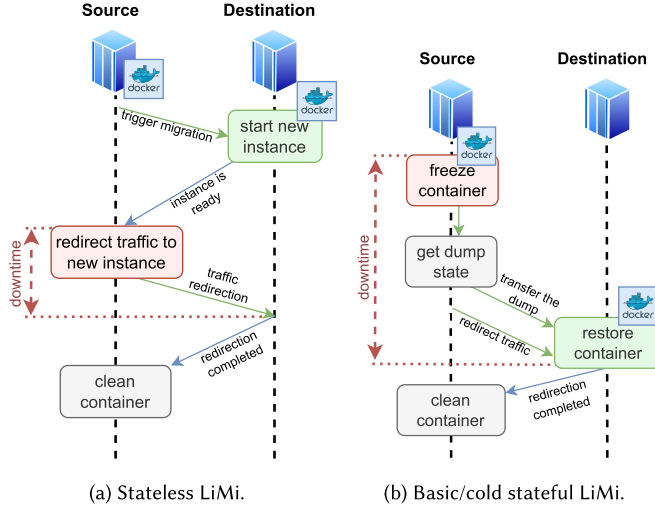


Fig. 4. Stateless vs. stateful service LiMi.

LiMi becomes much more accessible and efficient because there is no need to transfer the running instance and its volume. A new instance of the stateless service can be created at the destination node while the old instance is still running. Once this new instance is ready, the clients' traffic is redirected from the old instance to the new one. The relevant property of this method is the minimal service disruption that occurs only during the traffic redirection in a matter of milliseconds. In contrast, the downtime of stateful services includes the transfer time of the application state (comprising the VM or container volume) and the spin-up time for the application to start at the destination node, as depicted in Figure 4(b).

Another relevant property of stateless migrations, compared to stateful ones, is that the image used to start the new instance at the destination node is an original image of the service that can be guaranteed not to contain any malware or backdoor that was installed during a previous service lifetime cycle. That is to say, even when the previous instance of the service is unknowingly infected with an **advanced persistent threat (APT)** [44], after a stateless LiMi such infection is effectively removed from your service. This property alone makes stateless LiMi a highly required security feature for future networks, while stateful LiMi doesn't have this property and would transfer the APT along with the service's instance. Unfortunately, stateful services are the most common types of services and therefore a relevant research question is how to make this security property applicable in such circumstances. This challenge is further discussed in Section 6.1.

3.3 Stateful LiMi

Stateful services have a dynamic context and use data that frequently changes during its life cycle. Therefore, the standard method to live migrate them is to stop the instance (i.e., the VM or container), make a checkpoint of the instance's state, transfer this checkpoint to the destination node, and restore the instance from the checkpoint (as depicted in the Figure 4(b)). The main issue with this method is that the checkpointing of the service is performed during the *state dump* process. The state dump requires the instance to stop, implying that during the whole migration process, the service is not available to the end-users. This leads to a service downtime that depends on the service's instance size, checkpoint transfer speed, and instance restoration time. Thus, three optimization methods are traditionally used to solve this problem and reduce the service disruption:

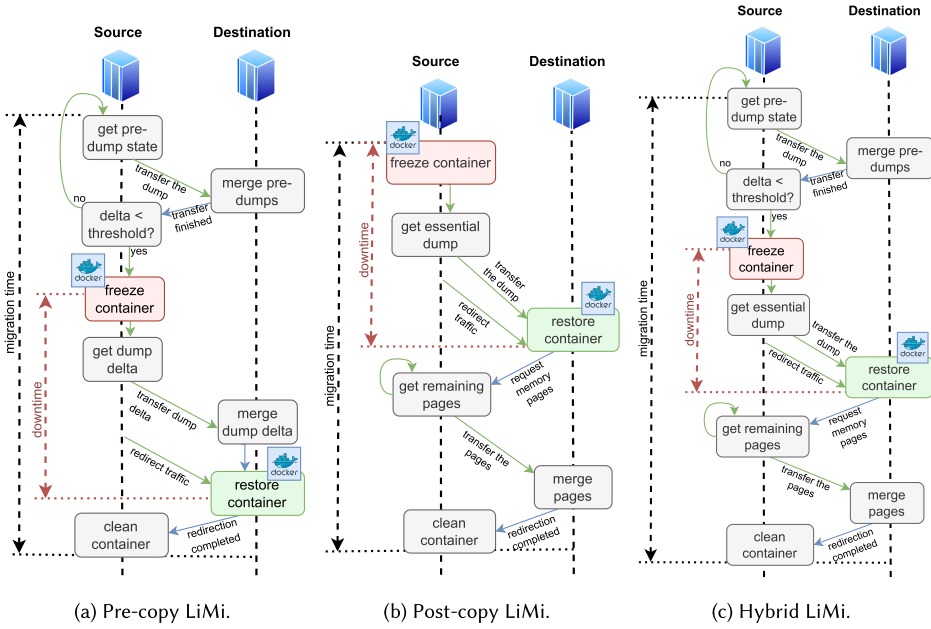


Fig. 5. Different stateful LiMi optimizations.

3.3.1 Pre-copy Optimization. The pre-copy or iterative optimization method is depicted in Figure 5(a) and consists of performing preliminary service state checkpoints, called *pre-dumps*, that do not require the service to be stopped [34]. As the service is still in use, the service state can change during the pre-dump, and the difference, named *delta*, is the only data that requires the service to freeze in order to be checkpointed and transferred with a *dump* call. The dumped *delta* is then merged with the pre-dumped images, allowing the restoration of the full instance at the destination node. To reduce the *delta* to its minimum, the pre-dump can be iterated several times before the final blocking dump. The iterative pre-dumps only consist of those memory pages that have been modified, also named *dirty pages*, that are added to the pages of previous pre-dumps [40]. The pre-dump iteration is configured by setting the maximum number of iterations wanted and the number of new dirty pages below which the blocking dump can be called to resume the migration. This configuration can be set and optimized with heuristics based on the service nature, as too many pre-dump iterations increase the overall migration time and network overhead, while a short pre-dump phase would instead increase the service downtime based on the frequency of memory changes in the service. Elsaid et al. provided a survey on the literature focusing on the pre-dump's parameters' optimization using mathematical models and heuristics [6]. In a separate study, Ma et al. [40] enhanced the pre-copy method to limit the iteration process to five iterations. This involves the exclusion of the most frequently updated memory pages from immediate copying, marking them to be copied only at the last round of the iteration process. Their method allows to reduce up to 34% of the total transferred data and up to 32.5% of the total migration time.

3.3.2 Post-copy Optimization. The post-copy or lazy migration method is another optimization method designed to reduce service downtime and is depicted in Figure 5(c). It first transfers with a blocking dump only the essential state that allows the instance to start at the destination node, such as the CPU state and the registers. Once the service runs and is available to users, the remaining memory pages are then injected into the running task address space in a phase called *pre-paging*,

Table 4. Inherited Properties of a Hybrid LiMi

Inheritance	Inherited from
Quick checkpoint/restore	post-copy
Smaller lazy pages delta	pre-copy
possibility of state corruption	post-copy
Long migration time	pre-copy and post-copy

Table 5. Evaluation and Comparison of Different LiMi Methods

Factors	basic	Pre-copy	Post-copy	Hybrid
Total migration time	lowest	low	low (except with throughput $\approx$ PDR)	highest
Service corruption	no	no	possible	possible
Downtime	high	low (except with throughput $\approx$ PDR)	low	low
QoS degradation	no	no	yes	yes
Total transferred data	low	high	low	middle

Table's sidenote: The basic and post-copy methods minimize total migration time transferring memory once, whereas pre-copy and hybrid methods involve repeated transfers of modified memory. Pre-copy and hybrid methods generally offer lower downtime at the cost of higher transferred data. Pre-copy is widely preferred in enterprise settings as Post-copy and hybrid methods may introduce service corruption and QoS degradation in low-throughput networks.

to restore the full-service state [45]. While the post-copy allows the downtime to be consistently reduced independently from the size or the update frequency of memory in the application, the QoS gets degraded when users' tasks need data that is still to be injected, moving from latency levels of RAM to considerably worst latency levels of network i/o operations.

3.3.3 Hybrid Optimization and Comparisons. Other hybrid optimization methods exist using both pre-copy and post-copy methods [41]. The objective of a hybrid method is to use the pre-copy to reduce the time of the pre-pages migration phase in order to minimize the exposure to a communication failure. At the same time, the post-copy would reduce the number of pre-dumps, making the total migration time less dependent on the frequency of memory changes in the application. However, as summarized in Table 4, the hybrid method also inherits both the drawbacks of pre-copy and post-copy, making it the method with the longest total migration time, as it has both a pre-dump and a pre-paging phase. The following subsection details the performance evaluations in the literature pointing out which methods to use based on the characteristics of the service to migrate, the network and infrastructure in place, and the requirements of the service.

3.3.4 Methods' Evaluation. Table 5 lists the differences between the four LiMi methods. One advantage of the basic and post-copy methods over the pre-copy and hybrid methods is that the memory pages are migrated only once during the migration. In contrast, with the pre-copy and hybrid methods, the modified memory pages have to be dumped/sent again during the pre-dump phase. This makes basic LiMi and post-copy LiMi faster to checkpoint and restore with a shorter total migration time and fewer data to transfer (i.e., reduced overhead over the network).

Puliafito et al. [15] evaluated the performance of these different LiMi methods on containerized services in two different fog network scenarios: a mobility scenario with very low throughput and high **round-trip time (RTT)** for node-to-node 4G/LTE LiMi, and a management/orchestration scenario using a bridged LAN with higher throughput and lower RTT. The authors tested each

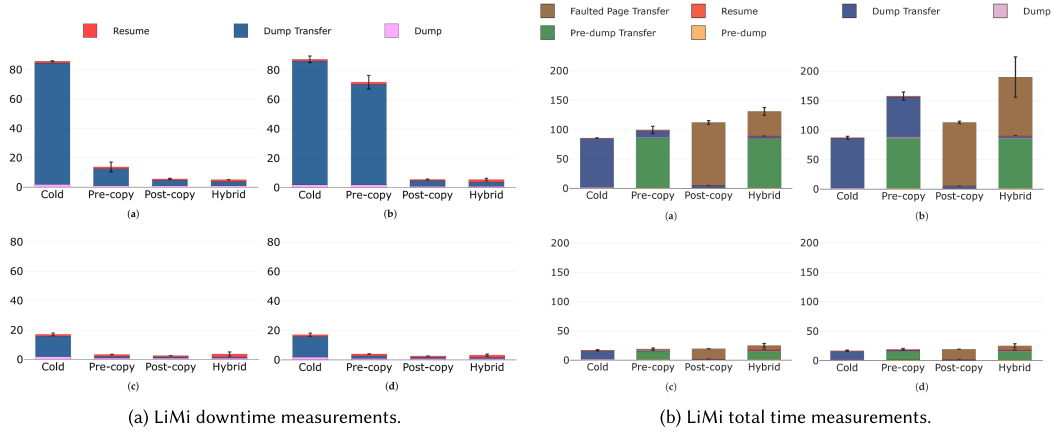


Fig. 6. Measurements of LiMi without optimization (*cold*) and with three optimization methods (pre-copy, post-copy, and hybrid) from Puliafito et al. work [15]. Configurations (a) and (b) present results in a 4G network with 11 Mbps of Throughput and 122 ms RTT. Configurations (c) and (d) present an average network with around 72 Mbps of throughput and 7 ms RTT. (a) and (c) have a low memory workload while (b) and (d) have a high memory workload.

scenario on the same runtime image size with high memory workload (i.e., high page dirtying rate) and low memory workload (i.e., low page dirtying rate), making for four different use cases, presented in Figure 6. Their results showed that the optimization methods have similar improvements in the downtime compared to the basic/cold migration. With the mobility scenario having a throughput as low as the page dirtying rate (around 11 Mbps), the pre-copy method loses its advantage over the basic migration as the dump phase transfers almost all the memory. This indicates that the post-copy and hybrid methods are more suitable in low-performance networks with memory-intensive applications. Nonetheless, this challenge is gradually diminishing with the continuous enhancement of current 5G networks and the anticipated advancements in future mobile networks. To illustrate this, an empirical evaluation was conducted with Swiss 5G connectivity using the Swisscom operator's network. The test involves establishing a 5G connection from a mobile device located in Winterthur to a Swisscom endpoint in Lausanne, covering a distance of 240 km. The test reveals a throughput of 660 Mbps with a latency of 21 ms. Compared to the mobile network performance used in [15], a substantial 60-fold increase was found in throughput, coupled with a five-fold decrease in latency, all while the distance between the two endpoints is a hundred kilometers longer.

Finally, the last phase of the pre-copy migration, the delta dump, is the only phase that affects the QoS of the migrated service. On the other hand, the post-copy method, while having a shorter downtime, worsens the application's QoS for a significantly longer period of time (during the transfer of remaining memory pages) and entails a critical risk of irreversible service corruption if communication fails during such phase. For this reason, the pre-copy optimization is the most used method in enterprise and production-ready ecosystems with established VM LiMi [46], as it transfers the totality of the instance's state before resuming the service, guaranteeing integrity and non-corruption of the service.

3.4 LiMi's Technical Implementation for VMs and Containers

VM LiMi is performed by the Hypervisor, or **Virtual Machine Monitor (VMM)**, which manages the full life-cycle of VMs in a virtualized environment. Major hypervisors, open-source and non,

Table 6. LiMi Optimization Work

Articles	VM/ Container	Method	Technology	Use Case	Objective
[37]	VM	Shared storage	Xen	Faster inter-cluster LiMi using NFS	Efficiency and QoS
[34]	VM	Pre-copy	Xen	Seamless inter-cluster LiMi of highly interactive apps	QoS
[40]	VM	Optimized pre-copy	Xen	Use of page marking to improve pre-copy LiMi	QoS
[6]	VM	Optimized pre-copy	Xen/KVM/ Qemu	Optimization of pre-copy parameters through cost modeling and heuristics	Efficiency and QoS
[45]	VM	Post-copy	Xen	LiMi of VMs in Gigabit LAN	Efficiency and QoS
[41]	VM	Hybrid	KVM/Qemu	LiMi of VMs in Gigabit LAN	Efficiency and QoS
[49]	VM	RDMA	Xen	Improving migration time using InfiniBand RDMA protocol	Performance and QoS
[50]	VM	Optimized pre-copy	KVM	Dirty-pages aware pre-copy LiMi of VMs in the edge over WAN	Performance and QoS
[51]	Container	Disk bypass	CRIU	Direct transfer of full state from source RAM to the destination RAM without intermediate disk storage as a buffer	Performance and QoS

Note: The technology reflects the system that has been used in respective articles to test the optimization method. Optimization methods per se are agnostic to technologies and could be implemented and applied to both VMs and containers, respectively.

such as KVM/QEMU, Xen, Virtioozzo, Hyper-V, and ESXi, implement LiMi and are the first systems on which the stateful optimizations described above are designed and tested (summarized in Table 6). These implementations are used by the major cloud platform providers such as AWS, Google, and Microsoft Azure and by private datacenters operating with popular open-source cloud platforms such as Openstack [47] and Proxmox [48].

CRIU is an open-source Linux-based library started by Virtioozzo, who also operates the VM hypervisor OpenVZ [52], and is a de-facto standard for container’s LiMi. CRIU is exclusively implemented in the user-space, ensuring its applicability across various Linux-based operating systems. It has evolved to become the de facto standard for container LiMi, integrated with major container management and orchestration systems. At the low level of a container runtime, such as runC and lxc, CRIU enables pre-copy, post-copy, and hybrid optimizations. At a higher container orchestration level, CRIU has been incorporated into Docker as a beta feature, not yet supporting the optimization algorithms such as pre-copy and post-copy. In contrast, LXD and Podman integrate CRIU with the pre-copy optimization, while a more recent integration of CRIU as a beta feature has been implemented on Kubernetes [53]. Kubernetes, arguably the most popular and used container orchestrator, has also presented a new method of live migrating containers running as StatefulSets in a Kubernetes environment [54]. As this is a recent unexplored method, it has yet to be compared with CRIU and evaluated in scientific literature. A preliminary study identifies this LiMi method as semi-stateful as the replication of StatefulSets only keeps the state of volumes, i.e., disk data, but does not transfer data in RAM and CPU registries as with CRIU [55].

CRIU’s checkpointing and restoring is introduced as an alpha feature starting from Kubernetes 1.25. While this feature does support migrating containers, the primary focus is on checkpointing for forensic purposes rather than full migration. This allows administrators to create an exact copy of a running container, enabling detailed analysis without disrupting the original container’s state.

Table 7. Traffic Relocation Work

Articles	VM/ Containers	Method	Tech	Use Case	Classification based on the taxonomy in Figure 7
[34]	VM	Layer-2 overlay	Xen	LAN LiMi	Inter-cluster, static-IP
[39]	VM	lightpath	Xen	Multi-cluster (MAN/WAN) adaptation of [34] on optical networks	Layer 3, static-IP
[56]	VM	IP tunneling and DynDNS	Xen	Multi-cluster LiMi using IP tunneling and DynDNS	Layer 3, dynamic-IP
[57]	Container	NAT	Kubernetes	Proximity-aware microservices' load-balancing	Layer 3, short-lived sessions
[58]	VM	MPTCP	KVM	VM WAN LiMi between cloudlets	Layer 4
[59]	VM	Mobile IPv6 standard	Xen	WAN VM LiMi	Layer 3
[60]	Container	SDN/P4/SRv6	CRIU	VM WAN live IPv6 edge workload migration	Layer 3
[61]	Container	Layer 7 (app)	CRIU	Live stream fog service	Layer 7
[62]	VM/ container	SDN	<i>n.n.</i>	Fixed TCP session's handover	Layer 3, long-lived sessions
[63]	VM/ container	TCP-repair	<i>n.n.</i>	Fixed TCP session's handover	Layer 4, long-lived sessions
[64]	VM/ container	2-socket proxy	<i>n.n.</i>	Handover of long-lived TCP/QUIC-based sessions	Layer 3, long-lived sessions

The checkpoint has to be restored by converting it first to an image or using the direct underlying commands from CRI-O. However, several limitations are noted in the current implementation:

- **Limited runtime support:** the CRIU feature is only available for specific container runtimes such as CRI-O (as it uses *runC*), but other container runtimes such as *Containerd* has its support still pending in the open source upstream repository.
- **Kubectrl integration:** there is no *kubectrl* support yet, so commands must be executed directly at the *kubelet* level.
- **No LiMi optimization methods:** pre-copy, post-copy, and hybrid optimizations are not supported, which limits its functionality in certain LiMi use cases with stringent QoS requirements.

One of the main security concerns discussed within the Kubernetes community is how to implement this feature safely. A checkpoint contains all the memory pages, including sensitive information like secrets. Therefore, it is crucial to enforce strict permissions for creating checkpoints and handling the checkpoint files to prevent unauthorized access or misuse. Further details on the security of LiMi are discussed in Section 5.4.1.

4 Connection Handover and Traffic Relocation in LiMi

Following the discussion on the background and classification of LiMi techniques, a key challenge to address is ensuring seamless connection handover and traffic relocation. This is critical for maintaining service availability and minimizing user disruption during migration. This redirection can be achieved through various methods, summarized in Table 7, all with the common goal of minimizing latency and QoS overhead. The choice of method depends on the ecosystem's properties and the migration scenario, as illustrated in Figure 7.

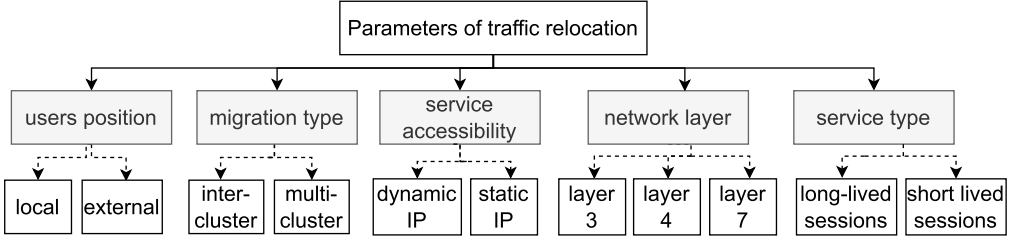


Fig. 7. Taxonomy of traffic relocation for LiMi.

These methods encompass:

- (1) *Service users' position*: this varies based on whether the users are connected externally (i.e., via the internet) or whether they are in the same local network or infrastructure of the application;
- (2) *Migration type*: this depends on whether the service is migrated within the same cluster or on a different one;
- (3) *Accessibility*: this depends on whether the IP of the service can be different after the migration or if it should keep the same IP for external accessibility;
- (4) *Network layer*: this factor depends on whether the connection handover of communication happens at the IP layer (i.e., layer 3 of the TCP/IP stack), transport layer (i.e., layer 4) or at the application layer (i.e., layer 7);
- (5) *Service type*: this depends on whether the service is using TCP short-lived sessions (e.g., HTTP requests or REST API requests) or TCP long-lived sessions (e.g., **Internet-of-Things (IoT)** devices, real-time applications, or network file transfers);

For service LiMi within the same cluster or LAN, Xen migrations provide the possibility of keeping the same IP address and performing the network migration at the link layer (i.e., layer 2) using ARP packets to update the IP and MAC address mapping in the order of tens of milliseconds [34]. For WAN migrations, the redirection has to happen at the transport layer or application layer. Most of the proxies implemented for load balancing at the transport layer make use of dynamic DNS, *iptables*, or Linux kernel's **IP Virtual Server (IPVS)** to perform **network address translation (NAT)**. However, these methods only work for new connection requests, while connections that are already established, i.e., with a session-based protocol such as TCP and QUIC, use the old IP of the service. This translates into a service disconnection when the old instance at the source is terminated, leading to service disruption or an ungraceful attempt to reconnect with the new IP address.

Bradford et al. [56] implemented a Xen VM migration mechanism that allows such live connections to reach the new instance using IP tunneling, while new connections find the new instance's IP with dynamic DNS. This redirection method incurs a service disruption of approximately 1.04 seconds, excluding the time for stateful migration. A potential drawback of this approach lies in the exposure of the new IP address change to end users. This visibility can be detrimental as it allows attackers to know when an instance is migrating and to discern the IP range used for the service shuffling. By exploiting this knowledge, targeted attacks could be launched that render defensive migrations ineffective. Moreover, IP tunneling requires the original source host to remain operational until all existing connections are terminated. While this poses minimal impact for web services with short-lived connections, the scenario is significantly different for long-lived connections commonly found in IoT and live streaming applications. In these cases, the extended

persistence of prior connections can lead to an unwanted increase in resource consumption as the source node can't be hibernated, as well as a decrease in the QoS because of the additional step in the data path to reach the new instance.

Fahs et al. [57] presented *Proxy-mity*, a proxy that replaces Kubernetes's standard kube-proxy for proximity-aware load balancing. The authors used *iptables*' NAT rules, which are limited to redirecting only new connection requests, while established connections keep using the old IPs and ports until they get disrupted by the instance termination in the source node.

Teka et al. [58] used **multipath TCP (MPTCP)** for seamless live VM migration between neighbor cloudlets. MPTCP allows simultaneous use of multiple IP addresses for one regular TCP session, an alternative solution to avoid re-establishment of the TCP connection. This approach hinges on two critical preconditions: (1) prior knowledge of the private IP address allocated to the new instance, and (2) enabling the services to use MPTCP. The latter requirement poses a significant challenge for cloud and telecommunication operators, as it necessitates control over the service binaries running on the machines. In these scenarios, the services might be owned by third-party clients, hindering the operator's ability to enforce MPTCP adoption.

Harney et al. [59] used Mobile IPv6 to keep the same IP address, avoiding changing the DNS, but with a long service disruption of 8 seconds (excluding the downtime due to the service migration itself). A more recent work in the same direction from Lemmi et al. [60] addressed IPv6 workload migration of an edge service keeping the same IP address by redirecting IPv6 traffic using SDN and state-of-the-art P4 network programmability [65], and the SRv6 protocol [66]. SRv6 follows the segment routing paradigm and adds a **segment routing header (SRH)** to the IPv6 header that allows to identify transit routers and dynamically performs IPv6 forwarding to migrate the network. Lemmi et al. additionally implement a buffer mechanism to keep packets sent during the CRIU container migration. Their results showed the best position to initiate the buffer is at the destination node, allowing a lossless traffic redirection while the service downtime due to CRIU's checkpoint/restore of their specific service is 3.28 seconds, which could be reduced by using an optimization method as described in Section 3.3.

Puliafito et al. [61] investigated stateless container-based service migration and the impact on the QoS perceived by mobile devices, achieving network downtimes of 5 s (in the experiment, 10 frames are lost in a 2 fps live stream fog service). The framework uses a white-box implementation where the connection handover happens at the application level, in the UE/end-device. This is not feasible in various use cases like cloud and telco operators who want to migrate resources independently of the application and the connected clients.

Binder et al. [62] used SDN to migrate live TCP connections, avoiding their re-establishment by synchronizing the TCP SEQ, congestion window, SYN/ACK numbers, and checksum value on the SDN switch at the time of packet forwarding. This "TCP state machine" synchronization is performed by changing all packets directed to the service from the moment the service is migrated until the connection of the client ends, which translates into an important SDN load and the QoS degradation that persists all along the connection life-cycle and is not limited to the first traffic redirection.

Cunha et al. [63] introduced a 2-socket proxy to redirect live attack connections to a Honeypot service. The proxy is introduced after the connection with the attacker is established using the Linux kernel's TCP_REPAIR feature, which is also used by CRIU when checkpointing a container runtime to save the container's network state. This work demonstrated that the 2-sockets redirection process outperforms SDN TCP handover and NFQUEUE packet mangling, with a continuous redirection latency overhead of only 0.03 ms. This comes with the condition of proxying only one specific connection to a specific port and needs to instantiate a different TCP_REPAIR proxy for singular ports and connections, which makes it inadequate for full VM or container LiMi.

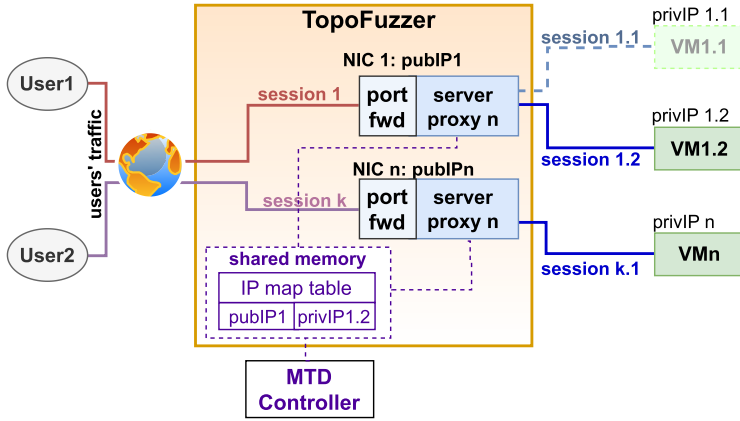


Fig. 8. TopoFuzzer long-lived session's redirection using two-socket proxies, from [64].

Soussi et al. presented Topofuzzer [64] which extends the 2-socket proxy approach to redirect all the communications of a migrating VM. Topofuzzer is tested in a 5G network by migrating stateless VNFs for proactive defense against malware infection and APTs. Depicted in Figure 8, Topofuzzer architecture is implemented using Mininet's Linux namespaces and **virtual network interfaces (vNICs)** as proxies to isolate the traffic redirection management of different VMs independently. It uses an in-memory hash table to map public service IPs to the dynamically changing local IPs during service migration. This table facilitates real-time updates to the proxy's backend socket, ensuring packet redirection to the correct destination. As Topofuzzer directs all traffic through a single proxy port, it leverages the Linux kernel's CONNTRACK functionality to recover the original packet ports. This introduces additional overhead for TCP compared to QUIC sessions, which benefit from immediate forwarding at the control plane due to the UDP protocol and NAT. The evaluation of Topofuzzer demonstrated a downtime of 330 ms for migrating stateless VMs, which corresponds to the traffic redirection time. For QUIC, the average downtime is 5 ms, as packets are immediately forwarded at the control plane level with NAT. Nonetheless, these results were measured on a limited test environment with a maximum of 100 connected users per migrated VM. Further testing is necessary to assess the scalability of the proposed solutions, which are also influenced by the capabilities of the physical **network interface cards (NICs)** used by the vNICs in the cloud environment.

5 Democratizing Container LiMi

This section delves into the literature on container and VM LiMi from the perspective of practical scenarios and use cases, encompassing three different themes and points of view: (1) the general work democratizing and extending the usage of LiMi (Section 5.1), especially for containers, in relevant use cases such as cloud ML training, TEEs, and advanced networks with particular protocols such as RDMA and SCTP; (2) LiMi in the different networking infrastructures, ranging from, microservices in the cloud (Section 5.2), to VNFs in telecommunication networks, discussing MEC and fog computing scenarios (Section 5.3); and (3) the security of LiMi and the usage of LiMi for security (Section 5.4).

5.1 Recent Advances on Key Use Cases

With CRIU now firmly established as the leading container runtime library for LiMi, recent research has shifted toward expanding its accessibility. This focus on democratizing container LiMi

Table 8. Works Enabling Container LiMi in Relevant use Cases

Articles	Use Case	Method	Technology
[67–70]	LiMi and cold start reduction of serverless containers	Container checkpointing	CRIU
[71]	LiMi of containers using RDMA protocol	Two new states to RDMA	RoCEv2/CRIU
[72]	LiMi of containers using QUIC protocol	Addition of QUIC redirection for server-side changes	CRIU
[73]	LiMi of containers using SCTP, specifically the 5G central unit	CRIU modification	CRIU
[74]	LiMi of containers using CUDA cores	Checkpoint/restore of CUDA API states checkpoint/restore	CRIU
[75–77]	LiMi of containers in TEEs	Secure transfer between two Intel SGX EPC enclaves	CRIU and SGX

aims at making it a viable option for deploying critical applications, some of which are covered in this section and summarized in Table 8.

5.1.1 Serverless Containers/applications. Serverless containers represent a fundamental aspect of the serverless computing paradigm, an emerging trend in cloud computing [78]. These containers enable applications to execute without requiring the management of underlying infrastructure or resources, thus simplifying the deployment process. In this sense, they are similar to lambda functions, where an application code defines how a container image is built (e.g., using Dockerfile for a Docker container) and deployed, or directly deploy a pre-built container image, leaving other tasks like scaling, load balancing, managing and maintaining the containers to the infrastructure/cloud provider. Providers such as AWS (AWS Fargate), Google (Google Cloud Run), and Microsoft (Azure container Instances) provide such type of service. Here, LiMi methods have to operate on containerized applications as a black box, which requires the migration to be feasible on any container runtime, which for technical reasons can be defined as any container built following the **Open Container Initiative (OCI)** standard specification [79]. With the increasing adoption of such types of applications, this requirement emphasizes the importance of democratizing and generalizing LiMi to all types of container applications.

Serverless containers mainly run stateless applications such as computer vision, data processing, monitoring, and RESTful APIs but could also run stateful applications such as web and mobile backends. Serverless applications could be event-driven and start by specific triggering events. In such common use case, different works [67–70] proposed to use LiMi’s checkpoint/restore algorithms to solve the problem of cold start delays of serverless applications. Their method, called *Prebaking*, consists of checkpointing the state of the running application with CRIU and using the dumped image to start new instances when an event triggers such action, saving multiple OS startup phases and **Just-In-Time (JIT)** compilation efforts used to improve the performance of high level languages such as Java, Javascript, and Python. Such a method decreases the starting time of serverless applications up to 25 times [67]. This method takes origin from the old use of process checkpointing at the operating system level to reduce process restarts following an execution failure [80, 81].

Rong et al. [82] performed the LiMi of video analytics applications using **convolutional neural networks (CNNs)**, such as vehicle counter, object tracking, and person detection. The authors presented a new algorithm other than the traditional checkpoint/restore, enabling the checkpoint

of GPU-enabled containers and tailored to the memory states' nature of a video analytics application, enabling LiMis in less than 405 ms. Yussupov et al. [83] also pointed out the lock-in problems faced when migrating serverless applications between multiple providers, due to the uncontrolled underlying system on which the containers run and the tighter-coupling with the providers' specific internal mechanisms: e.g., the events triggering the serverless application (i.e., what are the events and how are they triggered), and the integration with custom CI/CD pipelines. The authors proposed to use, among others, vendor-agnostic monitoring solutions and vendor-agnostic cloud modeling languages such as TOSCA [84].

5.1.2 RDMA. For instance, the current version of CRIU lacks support of the checkpoint/restore process for the **Remote Direct Memory Access (RDMA)** protocol, mainly because of the conflicting architectural principles of RDMA networks and containerization. Containers provide isolation between applications and the host, while RDMA networks aim at bringing applications and hardware closer together by transferring data to memory without involving the CPU or the operating system of the host. This significantly improves data transfer speed and reduces latency with SOTA InfiniBand NICs and switches capable of 400 Gbit/s bandwidth per port [85]. RDMA is broadly used in HPC, in clustered computing environments, and consequently in cloud infrastructures to enhance the efficiency of data transfers between nodes. Planeta et al. addressed the problem of the OS losing control over active RDMA connections by developing **MigrOS** [71], an architecture that adapt CRIU for the migration of containerized RDMA applications. MigrOS addresses this challenge by introducing two new states and two new message types to the RoCEv2 RDMA communication protocol, allowing for LiMi without interfering with critical path operations. This approach does not require modifications to user applications and maintains compatibility throughout the network stack. The authors claim to achieve up to 33% lower network latency compared to software-only techniques. In the LiMi of VM-based services that traditionally rely on the hypervisor and include unnecessary OS involvement, Huang et al. [49] leveraged the RDMA protocol and highspeed connectivity of the InfiniBand NIC, significantly improving the migration time and service downtime of up to 80% and 77%, respectively.

5.1.3 QUIC. Puliafito et al. [72] proposed minimal and non-breaking additions to QUIC to enable stateful LiMi of services using the protocol for long-lived communication. The authors tested their method on a stateful containerized application live migrated with the four different methods, namely: basic, pre-copy, post-copy, and hybrid methods. As this approach allows for the migrated service to change its IP address, authors compared it to redirection methods using dynamic DNS with TCP [56] and MPTCP [58] previously described. The LiMi with QUIC is shown to be more efficient. One note on this work is that the QUIC redirection TRIGGER starts at the beginning of the container migration, irrespective of the employed optimization method. This approach makes the application of pre-copy and/or hybrid downtime optimization ineffective, as the QUIC redirection interrupts the availability of the source instance, although during the iterative pre-dump phase the instance remains accessible and functional. Therefore, the findings presented in this article could be improved by modifying the timing of the redirection TRIGGER to start only at the dump phase. This would allow the QUIC connection to keep using the source instance of the service during the pre-dumps, reducing the migration downtime according to the pre-copy optimization method.

5.1.4 SCTP. Another critical protocol that lacks CRIU's checkpoint/restore support is the **Stream Control Transmission Protocol (SCTP)** protocol. SCTP is a transport layer connection-oriented protocol standardized in RFC 4,960 and widely used in telecommunication networks, especially in signaling protocols such as SIGTRAN [86], in **Voice over IP (VoIP)** applications, and in live stream applications. Ramanathan et al. [73] presented a proof-concept implementation that

enables the LiMi of a 5G **central unit (CU)**, containerized as a CNF, by modifying CRIU to support SCTP, used by the CU to exchange signaling messages with other 5G CNFs. The authors have experimentally validated their implementation in a federated testbed environment, employing the four LiMi methods and a VM-based CU LiMi for a comparative study. Their performance analysis concluded that the hybrid migration method decreases the downtime of the CU by up to 36% compared to the basic migration. Moreover, compared to state-of-the-art pre-copy VM migration, the container-based pre-copy migration reduces migration time by 96.7% and downtime by 75.2%.

5.1.5 GPUs. Eiling et al. [74] presented the Cricket framework, empowering CRIU to be used for the LiMi of services using GPU CUDA cores, notably used in machine learning applications and multimedia processing. This enables a more flexible GPU virtualization service in cloud computing and multi-tenant cluster usage. Cricket implementation emphasizes saving and restoring CUDA API-related execution states for the Checkpoint/Restore migration phases, integrating CRIU to checkpoint/restore the other non-GPU-related states. Performance evaluations show that the CUDA API states prolong the overall time of the restore phase at the destination node. The authors measure a downtime of 3.75 s with the basic method and consider the usage of incremental checkpointing (i.e., pre-copy and/or hybrid methods), as the downtime is deemed excessive unless there is a fault-tolerance requirement.

5.1.6 Trusted Execution Environments (TEEs). Liang et al. [75] presented a framework for the LiMi of containerized services running in TEEs, specifically, the migration from one Intel SGX enclave to another SGX. Enclave code and data are stored in the **Enclave Page Cache (EPC)**, which is isolated from normal memory and can only be accessed by code within SGX enclaves, enhancing confidentiality, integrity, and authentication. However, this introduces scalability and usability issues, including the ability to live migrate the containers. Previous works on live migrating SGX-enabled VMs have shown the tendency to create openings to fork/fallback attacks [76, 77], hindering the security benefits of the SGX enclave. At the same time, the need for migration between nodes to fully utilize cluster EPC memory is important and motivated by the scarcity of EPC resources in a single system, whose performance is impacted when overloaded with SGX-enabled containers. In their work, Liang et al. [75] showed that the restore time of containers with TEE enclaves is greater than the checkpoint time and grows exponentially with the increase of the memory size, compared to the linear increase for containers without a TEE enclave. For an application with a memory size of 128 MB, the migration time already increases by 60%, from 1.6 s to 4 s. This overhead comes from initializing the enclave at the destination and fetching the migration key. Intel has introduced its novel TEE technology for zero-trust remote VM execution, **Intel Trust Domain Extension (TDX)** [87], simplifying and generalizing the process of running VMs in an enclave compared to using lower-level SGX's API. The LiMi of **Trust Domains (TDs)**, the name given to TDX isolated VMs, is an ongoing area of development. Intel's **Migration TD (MigTD)** technology, outlined in their latest specification [88], serves as an open-source proof-of-concept and does not yet meet the production quality [89]. Similar to Intel's TDX, AMD's **Secure Encrypted Virtualization (SEV)** technology utilizes a TEE-based approach for VM protection. However, SEV relies on a **secure nested paging (SNP)** technique instead of EPC. SEV-SNP allows for VM migration using a centralized **Migration Agent (MA)** that manages and enforces security policies during the migration process [90].

5.2 LiMi in Microservices Architectures

In the context of microservices applications, the presence of interconnected containers introduces a heightened level of complexity for LiMi. Unlike the migration of standalone containers, the migration of multiple interdependent containers becomes a necessity due to their synchronous

relationship, constituting a service. Consequently, preserving the layout of these containers during migration becomes imperative.

Benjaponpitak et al. [42] addressed the imperative for enabling LiMi of these interconnected containers across diverse data centers and cloud providers. The primary focus of their work is the introduction of their fully automated LiMi solution, namely **CloudHopper**. This solution is tailored to facilitate the movement of Docker applications with minimal perceived downtime for clients within a multi-vendor, multi-cloud environment that spans across **Amazon Web Services (AWS)**, Google Cloud Platform, and Microsoft Azure cloud providers. The migration process in CloudHopper involves parallel and sequential processing of containers, commencing from the largest container and progressing to the smallest. This approach ensures the simultaneous completion of the migration for all containers. Results obtained through the utilization of the pre-copy optimization method revealed an average downtime of 7.5 seconds in scenarios where there is no workload or connected client. This downtime rises to 24.4 s when 400 clients are connected and the service is overloaded and involves **read/write (R/W)** disk operations. The factor increasing the downtime is the increased state dump to be transferred through the Internet, from a total of 21 MB for the scenario without workload to 1,165 MB for the workload scenario. However, in the scenario without a big dump state size, the decisive factor covering two-thirds of the migration downtime is the restore time, shown by their performance evaluation to have a constant average time of 5.4 seconds.

Velero, an open-source project developed and supported by VMWare [91], facilitates the secure backup and restoration of Kubernetes clusters, specifically designed for disaster recovery and the migration of full clusters. Velero supports multi-cluster and multivendor migration, such as moving a Kubernetes cluster from AWS to Microsoft Azure. However, Velero's migration does not qualify as LiMi, as it is not targeted for such use cases but rather for disaster recovery or occasional migrations. In contrast to tools like CRIU, Velero lacks the capacity for granular control over individual pod or container migrations. Additionally, CRIU can checkpoint RAM and CPU registries and offers a direct transfer of such data from the source's memory (i.e., the RAM) to the destination node's memory without intermediate disk storage as a buffer, a feature that significantly accelerates both checkpoint and restore tasks. This optimization was initially proposed in the literature by Venkatesh et al. [51] and later a similar in objective software was implemented by the CRIU upstream community as the *CRIU image streamer* [92].

KubeVirt [93] is a project that extends Kubernetes to support running and managing VMs alongside containerized workloads. This enables the usage of Kubernetes high-level abstraction of virtualized resources for the orchestration and management of both VMs and containers in the same cloud environment. KubeVirt allows for VM LiMi in Kubernetes using the pre-copy and the post-copy methods (the pre-copy being the default one). However, a constraint for KubeVirt VMs is to have a pod network binding to a bridge interface, meaning that the pod's network namespace containing the VM should not be associated with a network interface of the source host. The reason is that this would make the LiMi more complex, as the portability and compatibility of the workload across different Kubernetes nodes or clusters is reduced.

Overall, the LiMi of containers is still limited in microservices production-level environments with high-level orchestrators such as Kubernetes. The CRIU checkpoint/restore is integrated into Kubernetes only as a beta feature, while it still does not propose any optimization method to date (as of January 2024).

5.3 LiMi in Future Telecommunication Networks

MEC applies as a standard architecture for telecommunication networks starting from the advent of 5G and spanning to future 6G and nextG networks. It makes use of the concept of edge computing while also using networking technologies such as virtualization and software-defined

Table 9. LiMi in the Fog and MEC Context

Articles	VM/ Container	Method	Tech	Use Case	Objective
[42]	Containers	Ansible	CRIU	Multi-cloud Multi-vendor LiMi of interdependent container	Multi-vendor LiMi
[95]	Container	Stateless replicas	Kubernetes	CNF LiMi in an NFV-based 5G core	Edge LiMi
[96]	Container	basic LiMi	CRIU	Baseband Unit LiMi in LTE and LoRaWAN networks	Edge LiMi
[97]	VM/ containers	Pre-copy	KVM/LXC	Proximity-based edge LiMi	Proximity-aware LiMi
[50]	VM	Optimized pre-copy	KVM	Dirty-pages aware pre-copy LiMi of VMs in the edge over WAN	Edge-to-edge LiMi over WAN
[98]	VM	<i>n.n.</i>	<i>n.n.</i>	Location-aware VM migration in fog computing	Location-aware migration in Fog
[99]	Container	Post-copy	CRIU	Service migration in fog computing evaluating urban bus mobility patterns	LiMi in Fog
[100]	Container	Deep-RL	Docker	Optimal migration strategy in Fog	QoE and consumption in Fog
[101–105]	VM/ container	Swarm Intelligence	<i>n.n.</i>	Optimal service placement and migration strategy in Fog	Efficiency and QoS in Fog
[106]	Container	Genetic algorithm	<i>n.n.</i>	Inter-container communication of applications in Fog	QoS in Fog

networking to improve the availability of distributed telecommunication services [94]. The fog computing architecture, centered around small distributed data centers known as cloudlets, is designed to minimize application delays and processing times. Cloudlets, possessing lower computing capacity and proximity to users, can leverage larger cloud data centers when needed. In scenarios of user mobility, such as in mobile networks, this can further develop into trying to make the service follow the users. Similar to the concept of MEC and edge computing, fog computing fundamentally extends MEC by adding an intermediate layer in the network infrastructure spanning from the edge nodes to the centralized cloud data centers. This layer, also referred to as the “fog layer”, incorporates intermediate network nodes like routers and switches to provide computing resources, further pushing the distribution of computational tasks across the network. Table 9 summarizes the LiMi literature on MEC and fog computing analyzed in the following subsections.

5.3.1 LiMi on the Edge. Kaur et al. presented the LiMi of microservices between two remote Kubernetes clusters in the context of migrating CNFs of a Magma 5G core in an NFV-based TelcoCloud network [95]. This work, however, uses the stateless migration method starting the new instance in the destination node while the source node keeps running the older instance.

Schiller et al. [96] introduced a CRIU-based LiMi framework to migrate a running CNF. Specifically, the authors migrated the **Baseband Unit (BBU)** in two types of mobile networks, namely **Long Range Wide Area Network (LoRaWAN)** and **Long Term Evolution (LTE)** mobile networks. They observed that while the migration is possible within LoRaWAN networks, in the LTE network is not possible due to the BBU usage of the SCTP protocol in such networks. This issue is later addressed by Ramanathan et al. [73] (as detailed in Section 5.1.4) for live migrating the 5G CU CNF.

Machen et al. [97] presented a proximity framework to live migrate different services, both encapsulated in VMs (KVM) or system containers (LXC), from an edge node to another following

users' proximity. The authors used a layering approach similar to the pre-copy optimization to reduce the service downtime and present the advantages of live migrating containers over VMs for different types of services: i.e., face detection, video streaming, game server, RAM intensive app, and empty shell (i.e., container or VM). In the best case (the face detection service) and worst case (the RAM intensive service), containers had 29-fold and 4.7-fold shorter downtimes, respectively, compared to VMs.

Chaufournier et al. [50] tackled the issue of slow WAN networks when migrating a service from one edge to the other (e.g., via the Internet), worsening the downtime and availability of the service. They proposed a variation of the pre-copy optimization algorithm, namely *hot-skip*, tailored for edge cloud WAN that groups memory pages by historical write frequency and reduces migration time by 20% compared to KVM VM's LiMi. Authors additionally made use of MPTCP both to improve the LiMi time and the redirection of user traffic, which requires disclosing the migration and new IP addresses to the clients.

5.3.2 LiMi in the Fog. In this context, Bittencourt et al. [98] introduced a layered fog architecture for location-aware VM migration in fog computing, similar to the handover process of cellular networks. The work defines a general architecture providing information to tackle the optimization problem of LiMi, such as users' location, direction/ speed of movement, running applications, amount of stored data, data traffic needs, cloudlets capacity, network capacity, and users' usual behavior. Nonetheless, the methods to use such data and to make LiMi more efficient and less disruptive are not given.

Tang et al. [100] introduced an algorithm for the LiMi of containerized applications in Fog computing, addressing factors such as user **Quality-of-Experience (QoE)**, migration cost, and power consumption. The authors formulated the migration problem as a multi-dimensional **Markov Decision Process (MDP)**. Through the utilization of **deep reinforcement learning (deep-RL)**, they trained an agent to acquire an optimal migration strategy, demonstrating superior performance compared to established baseline approaches, particularly in terms of migration cost and energy efficiency. Given the multi-objective nature of this problem, an insightful extension would have been the use and benchmarking of **multi-objective MDP (MOMDP)** to model the migration and network observation, coupled with the use of **multi-objective deep-RL (MORL)**.

In another work, Puliafito et al. [99] introduced MobFogSim, a simulator that extends iFogSim [107] to model device mobility and containerized service migration in fog computing, evaluated with 100 urban bus mobility patterns derived from **Luxembourg SUMO Traffic (LuST)** dataset. The authors found that user speeds have a significant impact on the number of migrations. Users who move faster tend to experience fewer migrations, although the downtime of each migration is constant and unaffected by the user speed. They also found that cold migration is generally more efficient than post-copy migration, although the impact is not as significant as the impact of user speed. Puliafito et al. [15] also experimented with the different LiMi optimization methods on containerized services in a fog computing testbed, whose results and limitations have been already discussed in this survey in Section 3.3.4.

Gill et al. [101] proposed a different service placement approach based on the nature-inspired **Ant Colony Optimization (ACO)** algorithm, with the objective of minimizing the overall makespan of the task ensuring user QoS. Other similar works used and designed bio-inspired **Swarm Intelligence (SI)** algorithms for dynamic optimization problems in services and VM LiMi, especially in a cloud load balancing context as to decrease resource and energy consumption and the number of VM migrations while keeping or improving the QoS, measured in **service level agreement violation (SLAV)** [102–105].

Bourhim et al. [106] shifted the focus to the inter-container communication of applications in fog computing and used a near-optimal genetic algorithm based container placement strategy, another

Table 10. Works on Security and LiMi

Articles	VM/ Containers	Method	Tech	Use Case	Objective
[108]	VM	Firewall, IPS, IDS	Xen	Secure VM LiMi	Securing LiMi
[109]	VM	IPSec	Xen	Secure VM LiMi for automated load balancing	Securing LiMi
[110]	VM/ containers	SSH tunnel	KVM/LXC	Secure VM and container LiMi	Securing LiMi
[111, 112]	VM	vTPM	Xen	VM-vTPM LiMi	Securing LiMi
[113]	VM	SVM	KVM	Malware detection during VM LiMi	Securing LiMi
[114]	Containers	MTD	LXC	Evasive LiMi as a proactive MTD mechanism	LiMi for security
[22, 23]	Containers	MTD	Docker	MTD against vertical movements and border information leakage in public cloud	LiMi for security
[21]	VM	MTD	KVM	MTD against public cloud side channel attacks	LiMi for security
[115]	VM	MTD	KVM/ vSphere	LiMi as MTD against DDoS and Covert Channel Attacks	LiMi for security
[116]	VM	MOMDP and MORL	KVM	Multi-objective optimization of MTD strategy	LiMi for security, QoS, and efficiency

type of bio-inspired optimization algorithm falling in the category of evolutionary algorithms. In their proposed algorithm, different inter-container network communication modes are taken into account, namely host mode, overlay mode, and RDMA, to meet application response time requirements.

In summary, the efforts to refine LiMi techniques across both edge and fog environments used in telecommunication networks reflect a broader trend toward enhancing user experience and system efficiency. The varied approaches, ranging from AI/ML to bio-inspired algorithms, underscore the need for adaptable and efficient migration strategies that can handle the dynamic demands of modern networks, particularly in scenarios involving user mobility, enabling seamless service continuity even in highly variable conditions.

5.4 LiMi and Security

Table 10 summarizes the literature on LiMi's security analyzed. While Section 5.4.1 reviews methodologies proposed to guarantee integrity, data privacy, and authentication during LiMis, Section 5.4.2 explores the usage of LiMi for security as a defensive mechanism, covering both proactive and reactive strategies.

5.4.1 Securing LiMi. Multiple studies have been conducted securing the LiMi against network attacks such as **denial of service (DoS)**, man-in-the-middle, replay attacks, data theft, data corruption or tampering, and source/destination spoofing. These solutions are generally based on legacy communication security approaches.

for instance, Xianqin et al. [108] made use of firewalls, **intrusion prevention system (IPS)**, and **intrusion detection system (IDS)** in their solution for securing VM LiMis in a **network security enabled hypervisor (NSE-H)**.

Tamrakar et al. [109] proposed a framework using an IPSEC VPN tunnel for both authentication and cryptographically secure VM LiMi.

Mavuş et al. [110] used a similar approach with SSL for authentication with an application server, also bridging the service users to the running instance, and SSH for a secure data transfer through the encrypted communication channel. The authors measured and compared the performances of this method on different applications and encapsulated in both a VM (KVM) and a container (LXC), showing how the migration of the latter greatly outperforms the former in terms of migration time, service downtime, and data transfer (not because of a peculiarity in the proposed model but simply because of the difference between VMs and containers).

Danev et al. [111] enabled the secure migration of VMs leveraging **virtual Trusted Platform Modules (vTPM)**, a hardware-based protection of data and software. Their work proposes a vTPM key hierarchy with an intermediate layer between the physical TPM and the vTPM, enabling the implementation of a new secure protocol for VM-vTPM cold migration. This foundation paves the way for further enabling VM-vTPM LiMi. Building upon this, Peiru et al. [112] presented an improved vTPM-based integrity verification policy to ensure the authenticity and integrity of all entities involved (i.e., source, destination, and service) during secure VM-vTPM LiMi.

Watson et al. [113] presented a framework for malware detection during live VM migration. This framework analyzes the VM's checkpointed image for malicious code and network activity during the migration process. The authors claimed negligible impact on service downtime, migration time, and detection overhead due to the non-blocking nature of the analysis, which could detect an existing anomaly even after the migration has been concluded. This same feature of post-migration detection, nonetheless, makes this approach unsuitable for security scenarios where preventing infected VMs from entering a controlled environment is a priority over service availability.

The stability of live migrated services can be impacted during LiMi in some circumstances. For instance, containers that have their resource limits set with the Linux *cgroups* kernel functions (limits on CPU, RAM, and storage) could see their freezing time heavily affected when checkpointing the container for a pre-dump or dump image, sensitively decreasing the service availability. This problem is originated from the container runtime, e.g., *runc*, when the container is too limited, rather than CRIU failing the LiMi. Other cases where CRIU is involved occur when a container uses external mounts, writable root file system, external UNIX sockets, virtual Ethernet devices, shell jobs, or file locks. While CRIU can support most of these elements, it has to be carefully configured when migrating such containers. A misconfiguration of CRIU's checkpointing or restoring commands could lead to a failed migration, which without proper backup plans, results in losing the entirety of the running service. A usable CRIU feature that helps avoid such scenarios is to perform a LiMi that keeps the container running in the source after the migration, practically copying the container at the destination server and keeping the source copy in case the LiMi fails in any of its phases: checkpoint, transfer, or restore [117].

5.4.2 LiMi for Security. This set of works showcases the use of LiMi to improve service security and fault tolerance in light of new threats and risks of multi-tenant distributed cloud infrastructures.

One such application of LiMi is MTD, as introduced in Section 2.3. MTD hinges on the concept of dynamically altering the attack surface by relocating the target under attack. This dynamic movement disrupts the attacker's targeting process, hindering their ability to compromise the intended victim successfully. MTD is applied in a variety of cybersecurity topics other than networking, such as with **address space layout randomization (ASLR)** in software security [118] and with improving **machine learning (ML)** models' robustness against poisoning attacks [119, 120].

The adoption of MTD with LiMi has been explored to address various security threats. Azab et al. [114] quantified the probabilistic advantage of LiMi for MTD by considering scenarios where services are migrated across cloud hosts, some of which may be compromised. Their simulations

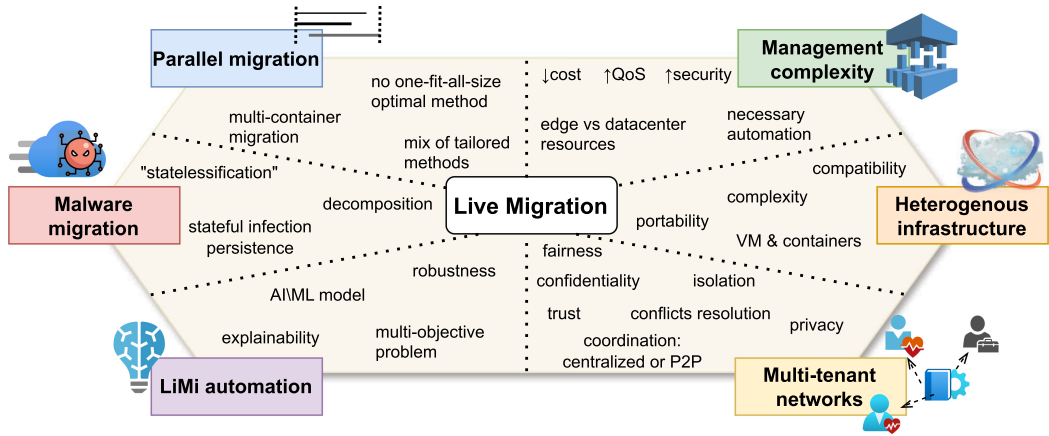


Fig. 9. Research challenges and directions for LiMi.

compared the effectiveness of MTD using LiMi when the number of compromised hosts is either static or dynamic, and they factor in IDS response time. The results demonstrated that container LiMi significantly reduces the probability of service compromise compared to a non-moving container.

Works like [22] and [23] leveraged MTD against vertical movements and border information leakage threats (e.g., side-channel attacks) that exploit isolation weaknesses in hypervisors managing VMs and containers. A similar approach proposed by Moon et al. [21], named NOMAD, employs VM migration based on a formal model of information leakage. This model considers factors that increase leakage rates, such as the number of service replicas and potential attacker collaboration.

Wang et al. [115] tackled both side-channel attacks and DDoS attacks on VMs and proposed an MTD framework made cost-effective by using a **renewal reward theory (RRT)**-based algorithm to evaluate the LiMi cost and determine when to perform the LiMi.

Soussi et al. [116] presented MERLINS, a security framework for proactive MTD in 5G networks via VNFs LiMi. They showed how MERLINS reactively mitigates malware infections and data corruption caused by a binary tampering attack on stateless VNFs. Proactively, MERLINS focuses on the multi-objective optimization problem of VNF LiMIs and reinstantiations, taking into consideration QoS improvement, security gains, and cost efficiency. For this, deep-RL is used to train an ML model for the MTD decisions based on the formal modeling of the 5G network into a multi-objective MOMDP. MERLINS uses SoTA MORL showcasing better higher performances compared to legacy single-objective deep-RL algorithms such as **proximal policy optimization (PPO)** and **advantage actor critic (A2C)**.

6 Challenges and Future Research Directions

While LiMi has shown considerable progress, its widespread adoption in future network architectures still faces several unresolved challenges. These challenges are categorized as in Figure 9 and further detailed afterward.

6.1 Malware Migration in Stateful vs. Stateless LiMi Security

A notable issue that has not been comprehensively addressed in the existing literature is related to the limitation in neutralizing malware infections and backdoors using stateful LiMi compared to the stateless LiMi method. In the case of stateless services, the image utilized for instantiating

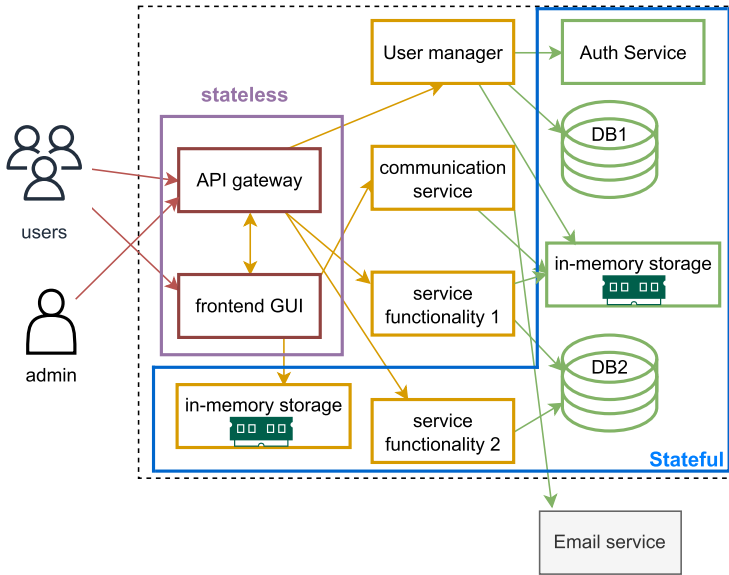


Fig. 10. Microservices generic architecture of a web service with stateless containers on the forefront of the service. The red containers are particularly exposed to potential security threats, given their reception of requests from external users/devices. Orange containers signify a diminished risk profile, whereas those in green exhibit the most minimal risk to security threats given the required attack chain to reach them.

a new instance at the destination node does not have to be a direct checkpoint of the running instance at the source node; instead, it can and should be derived from an authenticated, original container image free of any infection or malware introduced post-instantiation. Conversely, in the context of stateful services, if an instance is contaminated and subsequently migrated, the new instance inherits the infection, perpetuating the compromised state of the service, i.e., stateful infection persistence. This distinction underscores a critical security difference and a challenge in maintaining the integrity and security of stateful services.

To use, at best, the fading security property of stateless LiMi when performing LiMi of stateful services, a solution is to leverage the microservices architecture and already established decomposition best practices. One such best practice in decomposing the service into multiple microservices is to separate the stateless components of the service into a maximized set while maintaining a minimal set for stateful components, which may reside in separate microservices (or more traditional server VMs based on specific technical requirements), i.e., a “stateless-ification”.

Illustrated in Figure 10 is an example application of this architecture for a generic web service comprising of a front-end **graphical user interface (GUI)**, a user management and authentication system, a communication and message/queue service application, and other potential stateful applications requiring a database and/or an in-memory cache. The proposed security solution involves granting access to the stateful microservices through the stateless microservices, such that compromising a stateful container would require first to breach and compromise the stateless container. This configuration facilitates more frequent stateless LiMis for containers at the service forefront, which are less critical but most susceptible to attacks and undetected infections—akin to a demilitarized zone in a networked system. This approach enables the frequent “*cleaning*” of the service from potential malware and backdoors, significantly reducing the likelihood of attackers advancing to the stateful critical containers sitting in the back. Furthermore, this setup would limit

the necessity of frequent stateful LiMIs of the latter containers, not used for infection cleansing but as a proactive MTD strategy to reduce the ASP of side-channel attacks as previously discussed in Section 5.4.2.

6.2 Parallel Migration of Microservices for a Single Application

Existing research addresses the challenge of migrating multiple containers or VMs composing a single application simultaneously. This is especially the case with the microservices architectures. However, major limitations exist:

- VM-centric migration: Some approaches assume containers of the same application reside on a single VM, facilitating LiMi of the entire VM [93]. This is not applicable when containers are scattered across different VMs.
- Heterogeneous microservices load: Other studies do explore parallel individual container migration, but they employ the same LiMi method to all containers [42]. This approach fails to consider the diverse nature and resource demands of each container. In fact, those can be memory-intensive, CPU-intensive, network-intensive (i.e., high I/O), or disk-intensive (i.e., high R/W).

As the survey reveals the lack of a one-fits-all-size optimal LiMi method for all types of load, a dynamic approach that tailors the migration algorithm to the specific container type is necessary. For instance, microservices commonly utilize standalone containers for caches and dictionaries in in-memory data structures (e.g., Redis and Memcached). In such scenarios, in-memory LiMi techniques like VAS-CRIU or CRIU image streamer ([51, 92]) can be applied, bypassing disk storage buffers. Similarly, microservices with high dirty page rates can benefit from a simpler cold migration approach without additional optimization methods, or, for smaller stateful containers, a post-copy migration strategy might incur lower QoS overhead compared to pre-copy alternatives when trying to lower the service downtime.

Hence, this work advocates for research into frameworks that leverage the microservice architecture and employ container-specific tailored methods, involving the mixed use of optimization techniques to improve the performance of the overall LiMi system.

6.3 Heterogeneous Infrastructure

LiMi necessitates compatibility management between differing microprocessor architectures, operating systems, applications, and resource requirements of the migrating entities. For instance, directly migrating an Intel TDX VM to an AMD SEV-SNP VM is not currently feasible due to fundamental differences in isolation techniques. On the container runtime side, the CRIU library is currently limited to Linux environments, hindering its application on Windows or MacOS systems. Moreover, high-level container orchestrators like Kubernetes and Docker support CRIU integration as an alpha/beta feature and require the addition of optimization methods to fully leverage its capabilities.

Another future research direction also relevant to the democratization of LiMi is to extend LiMi applicability to **large language models (LLMs)**. LLMs have substantial infrastructure requirements, such as single system supercomputers [121], which makes them heavily reliant on the physical hardware they are deployed on. For this reason, LLMs generally run directly on physical servers and rarely on containers. Another significant challenge is the sheer size of these models. For example, the GPT-3 model necessitates around 350 GB of storage just for the model parameters (two bytes per parameter across 175 billion parameters). Meanwhile, the state-of-the-art open-source model Llama 3.1, with its 405 billion parameters, demands approximately 820 GB of

storage, at least 1 TB of RAM, and ideally GPU VRAM, with 640 GB recommended across multiple high-end GPUs to optimize inference speed [122]. Although Llama 3.1 also offers smaller variants with 8 billion and 70 billion parameters, these models present similar challenges on a reduced scale. To the best of the author's knowledge, there is no published research or disclosed work on LLM's LiMi, suggesting that such an endeavor may still be beyond current technological capabilities. Nevertheless, it is anticipated that as the need for more flexible deployment schemes grows, LLM providers such as OpenAI, Microsoft, and Google will seek solutions to migrate LLMs across their larger and more capable cloud infrastructures.

6.4 Management Complexity

Likewise, LiMi of VMs and containers across geographically dispersed locations introduces significant management complexity. This complexity encompasses not only the migration process itself, but also the ongoing management of services on heterogeneous resources with varying capabilities. Factors like resource-constrained edges, diverse network characteristics (impacting migration time and downtime), and inconsistent security/confidentiality guarantees further exacerbate these challenges. The sheer volume and intricacy of these challenges makes manual management untenable and prone to errors. Consequently, research in cognitive and automated management/orchestration for LiMi is essential. A key aspect of such cognitive systems is their ability to optimize and adapt to diverse situations based on network type, service type, migration type, and migration objectives: improved QoS/QoE, energy consumption, carbon footprint, proactive security, and operational costs. This necessitates research into multi-objective optimization problems, particularly within large, high-dimensional spaces with multiple competing objectives.

6.5 Explainability and Robustness of LiMi Automation

The automation of network management tasks with cognitive decision-making capabilities necessitates explainable decision-making. This requirement is particularly crucial in critical infrastructures where operators manage services owned by third parties. Explainability ensures human oversight aligns with liability and **service level agreements (SLAs)**. The emergence of **neural network (NN)**-based ML for network management introduces a significant challenge due to their inherent lack of interpretability without additional explainability mechanisms. When compared to traditional heuristics, formal models, or genetic/swarm intelligence approaches, the adoption of NNs must be accompanied by justification and a minimum level of guaranteed explainability.

Furthermore, automated decision systems must exhibit robustness against adversarial inputs. This is critical, for instance, to prevent attackers from triggering LiMis, either for their own instances or those of other tenants. Such attacks could aim at disrupting service availability for targeted tenants (DoS attack) or manipulate resource consumption, impacting overall infrastructure performance and affecting other tenants as well.

6.6 Multi-tenant Networks

LiMi tasks and policies inherently introduce complexity in multi-tenant network infrastructures. These policies must address critical aspects such as confidentiality, trust, and fairness between tenants. Improperly defined migration policies can negate the isolation benefits discussed in Section 2. For instance, in a TEE-protected environment, secure and verifiable attestation (proof of identity) must be preserved all along the phases of the migration. In the scope of autonomous decision systems, a LiMi policy framework becomes necessary to determine who has decision control and whether a single tenant (i.e., a centralized structure) or all tenants (i.e., a peer-to-peer structure) exert control. In the latter case, rogue tenants could abuse service migration, raising

security concerns. A zero-trust approach (e.g., leveraging TEEs) is crucial to minimize such hazards. Moreover, each tenant could use an internal policy control system that validates the LiMi decisions made for its services with the influence of external tenants. As an alternative, tenants could employ independent decision systems that only share data and network intelligence with others to improve informative LiMi decisions. This is nonetheless not trivial, as it requires careful design of the information sharing system to preserve tenants' privacy.

7 Conclusions

This survey provides a comprehensive overview of LiMi techniques, encompassing network considerations, downtime reduction strategies, and the technical distinctions between VM and container LiMi. It highlights recent advancements that democratize LiMi for containerized workloads. Tools like CRIU have been extended to support previously incompatible but critical corner cases, such as containers in TEEs for confidential computing or CUDA cores for AI/ML applications. Additionally, the survey explores methods to ensure LiMi security against threats as well as using LiMi as a security mechanism itself. Open challenges are identified, including managing heterogeneous environments, optimizing multi-container LiMIs in microservices, and enhancing stateful services' security.

The survey concludes that LiMi is a critical technology with growing importance in future networks, including telecommunication networks, multi-cloud environments, and edge computing infrastructures. Advances in LiMi optimization techniques, the proliferation of technologies like containers, and the adoption of intelligent decision-making algorithms have significantly reduced the overhead associated with LiMi, making it applicable and feasible even for scenarios with stringent availability requirements.

References

- [1] Diego Kreutz, Fernando M. V. Ramos, Paulo Esteves Verissimo, Christian Esteve Rothenberg, Siamak Azodolmolky, and Steve Uhlig. 2014. Software-defined networking: A comprehensive survey. *Proc. IEEE* 103, 1 (2014), 14–76.
- [2] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, et al. 2014. P4: Programming protocol-independent packet processors. *ACM SIGCOMM Computer Communication Review* 44, 3 (2014), 87–95.
- [3] Rashid Mijumbi, Joan Serrat, Juan-Luis Gorricho, Niels Bouten, Filip De Turck, and Raouf Boutaba. 2015. Network function virtualization: State-of-the-art and research challenges. *IEEE Communications Surveys and Tutorials* 18, 1 (2015), 236–262.
- [4] Luiz Bittencourt, Roger Immich, Rizos Sakellariou, Nelson Fonseca, Edmundo Madeira, Marilia Curado, Leandro Villas, Luiz DaSilva, Craig Lee, and Omer Rana. 2018. The internet of things, fog and cloud continuum: Integration and challenges. *Internet of Things* 3 (2018), 134–155. <https://www.sciencedirect.com/journal/internet-of-things/vol/3/suppl/C>
- [5] Heleen L. Van Soest, Michel G. J. den Elzen, and Detlef P. van Vuuren. 2021. Net-zero emission targets for major emitting countries consistent with the Paris Agreement. *Nature Communications* 12, 1 (2021), 2140.
- [6] Mohamed Esam Elsaid, Hazem M. Abbas, and Christoph Meinel. 2022. Virtual machines pre-copy live migration cost modeling and prediction: A survey. *Distributed and Parallel Databases* 40, 2–3 (2022), 441–474. DOI: <http://dx.doi.org/10.1007/S10619-021-07387-2/FIGURES/7>
- [7] Fei Zhang, Guangming Liu, Xiaoming Fu, and Ramin Yahyapour. 2018. A survey on virtual machine migration: Challenges, techniques, and open issues. *IEEE Communications Surveys and Tutorials* 20, 2 (2018), 1206–1243. DOI: <http://dx.doi.org/10.1109/COMST.2018.2794881>
- [8] Violeta Medina and Juan Manuel García. 2014. A survey of migration mechanisms of virtual machines. *ACM Comput. Surv.* 46, 3 (2014), 33 pages. DOI: <http://dx.doi.org/10.1145/2492705>
- [9] Divya Kapil, Emmanuel S. Pilli, and Ramesh C. Joshi. 2013. Live virtual machine migration techniques: Survey and research challenges. In *Proceedings of the 2013 3rd IEEE International Advance Computing Conference*. 963–969. DOI: <http://dx.doi.org/10.1109/IAdCC.2013.6514357>

- [10] Mostafa Noshay, Abdelhameed Ibrahim, and Hesham Arafat Ali. 2018. Optimization of live virtual machine migration in cloud computing: A survey and future directions. *Journal of Network and Computer Applications* 110 (2018), 1–10. <https://www.sciencedirect.com/journal/journal-of-network-and-computer-applications/vol/110/suppl/C>
- [11] Shangguang Wang, Jinliang Xu, Ning Zhang, and Yujiong Liu. 2018. A survey on service migration in mobile edge computing. *IEEE Access* 6 (2018), 23511–23528. DOI: <http://dx.doi.org/10.1109/ACCESS.2018.2828102>
- [12] Muhammad Imran, Muhammad Ibrahim, Muhammad Salah Ud Din, Muhammad Atif Ur Rehman, and Byung Seo Kim. 2022. Live virtual machine migration: A survey, research challenges, and future directions. *Computers and Electrical Engineering* 103 (2022), 108297. <https://www.sciencedirect.com/journal/computers-and-electrical-engineering/vol/103/suppl/C>
- [13] Tianzhang He and Rajkumar Buyya. 2023. A taxonomy of live migration management in cloud computing. *ACM Comput. Surv.* 56, 3, Article 56 (oct 2023), 33 pages. DOI: <http://dx.doi.org/10.1145/3615353>
- [14] Zeineb Rejiba, Xavier Masip-Bruin, and Eva Marín-Tordera. 2019. A survey on mobility-induced service migration in the fog, edge, and related computing paradigms. *ACM Computing Surveys* 52, 5 (2019), 33 pages. DOI: <http://dx.doi.org/10.1145/3326540>
- [15] Carlo Puliafito, Carlo Vallati, Enzo Mingozzi, Giovanni Merlino, Francesco Longo, and Antonio Puliafito. 2019. Container migration in the Fog: A performance evaluation. *Sensors* 19, 7 (2019). <https://www.mdpi.com/1424-8220/19/7/1488>
- [16] Kiranpreet Kaur, Fabrice Guillemin, and Francoise Sailhan. 2022. Container placement and migration strategies for cloud, fog, and edge data centers: A survey. *International Journal of Network Management* 32, 6 (2022), e2212. DOI: <http://dx.doi.org/10.1002/NEM.2212>
- [17] OpenVZ. Checkpoint/Restore In Userspace - CRIU. Retrieved January 10, 2024 from <https://criu.org>. ([n. d.]).
- [18] Ali Bakhoda, George L. Yuan, Wilson W. L. Fung, Henry Wong, and Tor M. Aamodt. 2009. Analyzing CUDA workloads using a detailed GPU simulator. In *Proceedings of the 2009 IEEE International Symposium on Performance Analysis of Systems and Software*. IEEE, 163–174.
- [19] Mohamed Sabt, Mohammed Achemlal, and Abdelmadjid Bouabdallah. 2015. Trusted execution environment: What it is, and what it is not. In *Proceedings of the IEEE Trustcom/BigDataSE/ISpa*.
- [20] Tarik Taleb, Konstantinos Samdanis, Badr Mada, Hannu Flinck, Sunny Dutta, and Dario Sabella. 2017. On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration. *IEEE Communications Surveys and Tutorials* 19, 3 (2017), 1657–1681. DOI: <http://dx.doi.org/10.1109/COMST.2017.2705720>
- [21] Soo-Jin Moon, Vyas Sekar, and Michael K. Reiter. 2015. Nomad: Mitigating arbitrary cloud side channels via provider-assisted migration. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA. DOI: <http://dx.doi.org/10.1145/2810103.2813706>
- [22] Rui Huang, Hongqi Zhang, Yi Liu, and Shie Zhou. 2017. RELOCATE: A container based moving target defense approach. In *Proceedings of The 7th International Conference on Computer Engineering and Networks — PoS (CENet2017)* 299 (2017), 008. DOI: <http://dx.doi.org/10.22323/1.299.0008>
- [23] Mohamed Azab and Mohamed Eltoweissy. 2016. MIGRATE: Towards a lightweight moving-target defense against cloud side-channels. *Proceedings - 2016 IEEE Symposium on Security and Privacy Workshops, SPW 2016* (8 2016), 96–103. DOI: <http://dx.doi.org/10.1109/SPW.2016.28>
- [24] Paul Voigt and Axel Von dem Bussche. 2017. The EU general data protection regulation (GDPR). *A Practical Guide, 1st Ed., Cham: Springer International Publishing* 10, 3152676 (2017), 10–5555.
- [25] NIST. 2024. NIST Glossary: Moving Target Defense. Retrieved 21 November 2024 from https://csrc.nist.gov/glossary/term/moving_target_defense. (Jan 2024).
- [26] Shanshan Li, He Zhang, Zijia Jia, Chenxing Zhong, Cheng Zhang, Zhihao Shan, Jinfeng Shen, and Muhammad Ali Babar. 2021. Understanding and addressing quality attributes of microservices architecture: A Systematic literature review. *Information and Software Technology* 131 (2021), 106449. <https://www.sciencedirect.com/journal/information-and-software-technology/vol/131/suppl/C>
- [27] Tarik Taleb, Sunny Dutta, Ksentini Adlen, Muddesar Iqbal, and Flinck Hannu. 2017. Mobile edge computing potential in making cities smarter. *IEEE Communications Mag.* 55, 3 (2017), 38–43. DOI: <http://dx.doi.org/10.1109/MCOM.2017.1600249CM>
- [28] Tao Ouyang, Zhi Zhou, and Xu Chen. 2018. Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing. *IEEE Journal on Selected Areas in Communications* 36, 10 (2018), 2333–2345. DOI: <http://dx.doi.org/10.1109/JSAC.2018.2869954>
- [29] Shiqiang Wang, Rahul Urgaonkar, Murtaza Zafer, Ting He, Kevin Chan, and Kin K. Leung. 2019. Dynamic service migration in mobile edge computing based on markov decision process. *IEEE/ACM Transactions on Networking* 27, 3 (2019), 1272–1288. DOI: <http://dx.doi.org/10.1109/TNET.2019.2916577>
- [30] International Energy Agency (IEA). 2024. Data Centres and Data Transmission Networks. Retrieved 21 November 2024 from <https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks>. (2024).

- [31] Fabien Hermenier, Nicolas Lorient, and Jean-Marc Menaud. 2006. Power management in grid computing with xen. In *Proceedings of the International Symposium on Parallel and Distributed Processing and Applications*. Springer, 407–416.
- [32] Gong Chen, Wenbo He, Jie Liu, Suman Nath, Leonidas Rigas, Lin Xiao, and Feng Zhao. 2008. Energy-aware server provisioning and load dispatching for connection-intensive internet services. In *Proceedings of the NSDI*. 337–350.
- [33] Qiang Huang, Fengqian Gao, Rui Wang, and Zhengwei Qi. 2011. Power consumption of virtual machine live migration in clouds. In *Proceedings of the 2011 3rd International Conference on Communications and Mobile Computing*. 122–125. DOI : <http://dx.doi.org/10.1109/CMC.2011.62>
- [34] Christopher Clark, Keir Fraser, Steven Hand, Jacob Gorm Hansen, Eric Jul, Christian Limpach, Ian Pratt, and Andrew Warfield. 2005. Live migration of virtual machines. In *Proceedings of the 2nd Conference on Symposium on Networked Systems Design and Implementation-Volume 2*. 273–286.
- [35] William Voorsluys, James Broberg, Srikanth Venugopal, and Rajkumar Buyya. 2009. Cost of virtual machine live migration in clouds: A performance evaluation. In *Cloud Computing: 1st International Conference, CloudCom 2009, Beijing, China, December 1–4, 2009. Proceedings 1*. Springer, 254–265.
- [36] Haikun Liu, Hai Jin, Xiaofei Liao, Liting Hu, and Chen Yu. 2009. Live migration of virtual machine based on full system trace and replay. In *Proceedings of the 18th ACM International Symposium on High Performance Distributed Computing*. 101–110.
- [37] Changyeon Jo, Erik Gustafsson, Jeongseok Son, and Bernhard Egger. 2013. Efficient live migration of virtual machines using shared storage. *ACM Sigplan Notices* 48, 7 (2013), 41–50.
- [38] Michael R. Hines, Umesh Deshpande, and Kartik Gopalan. 2009. Post-copy live migration of virtual machines. *ACM SIGOPS Operating Systems Review* 43, 3 (2009), 14–26.
- [39] Franco Travostino, Paul Dasptit, Leon Gommans, Chetan Jog, Cees De Laat, Joe Mambretti, Inder Monga, Bas Van Oudenaarde, Satish Raghunath, and Phil Yonghui Wang. 2006. Seamless live migration of virtual machines over the MAN/WAN. *Future Generation Computer Systems* 22, 8 (2006), 901–907.
- [40] Fei Ma, Feng Liu, and Zhen Liu. 2010. Live virtual machine migration based on improved pre-copy approach. In *Proceedings of the 2010 IEEE International Conference on Software Engineering and Service Sciences*. IEEE, 230–233.
- [41] Shashank Sahni and Vasudeva Varma. 2012. A hybrid approach to live migration of virtual machines. In *Proceedings of the 2012 IEEE International Conference on Cloud Computing in Emerging Markets*. IEEE, 1–5.
- [42] Thad Benjaponpitak, Meatasit Karakate, and Kunwadee Sripanidkulchai. 2020. Enabling live migration of containerized applications across clouds. In *Proceedings of the IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2529–2538.
- [43] Sai Venkat Naresk Kotikalapudi. 2017. *Comparing Live Migration between Linux Containers and Kernel Virtual Machine: Investigation Study in Terms of Parameters*. .
- [44] Adel Alshamrani, Sowmya Myneni, Ankur Chowdhary, and Dijiang Huang. 2019. A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. *IEEE Communications Surveys and Tutorials* 21, 2 (2019), 1851–1877.
- [45] Michael R. Hines, Umesh Deshpande, and Kartik Gopalan. 2009. Post-copy live migration of virtual machines. *ACM SIGOPS Operating Systems Review* 43, 3 (2009), 14–26. DOI : <http://dx.doi.org/10.1145/1618525.1618528>
- [46] Tuan Le. 2020. A survey of live Virtual Machine migration techniques. *Computer Science Review* 38 (2020), 100304. <https://www.sciencedirect.com/journal/computer-science-review/vol/38/suppl/C>
- [47] OpenStack Foundation. 2024. OpenStack. (2024). Retrieved April 04, 2024 from <https://www.openstack.org/>
- [48] Proxmox VE. 2024. Proxmox Virtual Environment. (2024). Retrieved April 04, 2024 from <https://www.proxmox.com/en/proxmox-ve>
- [49] Wei Huang, Qi Gao, Jiuxing Liu, and Dhabaleswar K. Panda. 2007. High performance virtual machine migration with RDMA over modern interconnects. In *Proceedings of the 2007 IEEE International Conference on Cluster Computing*. 11–20. DOI : <http://dx.doi.org/10.1109/CLUSTER.2007.4629212>
- [50] Lucas Chaufournier, Prateek Sharma, Franck Le, Erich Nahum, Prashant Shenoy, and Don Towsley. 2017. Fast transparent virtual machine migration in distributed edge clouds. In *Proceedings of the 2nd ACM/IEEE Symposium on Edge Computing*. Association for Computing Machinery, New York, NY, USA, 13 pages. DOI : <http://dx.doi.org/10.1145/3132211.3134445>
- [51] Ranjan Sarpangala Venkatesh, Till Smejkal, Dejan S. Milojicic, and Ada Gavrilovska. 2019. Fast in-memory CRIU for docker containers. In *Proceedings of the International Symposium on Memory Systems*. Association for Computing Machinery, New York, NY, USA, 53–65. DOI : <http://dx.doi.org/10.1145/3357526.3357542>
- [52] OpenVZ. 2024. OpenVZ. Retrieved April 04, 2024 from <https://www.openvz.org/>. (2024).
- [53] Adrian Reber (Red Hat). Forensic Container Checkpointing in Kubernetes. ([n. d.]). Retrieved August 14, 2024 from <https://kubernetes.io/blog/2022/12/05/forensic-container-checkpointing-alpha/>

- [54] Peter Schuurman (Google). Kubernetes 1.27: StatefulSet Start Ordinal Simplifies Migration. ([n. d.]). Retrieved August 14, 2024 from <https://kubernetes.io/blog/2023/04/28/statefulset-start-ordinal/>
- [55] Anthony Mamaril, Rinchen Kolodziejczyk, Wissem Soussi, and Gurkan Gur. 2024. Exploring live payload migrations for MTD in microservices architecture. In *Proceedings of the IEEE 99th Vehicular Technology Conference (VTC2024-Spring)*. 1–5. DOI : <http://dx.doi.org/10.1109/VTC2024-Spring62846.2024.10683429>
- [56] Robert Bradford, Evangelos Kotsovinos, Anja Feldmann, and Harald Schiöberg. 2007. Live wide-area migration of virtual machines including local persistent state. In *Proceedings of the 3rd International Conference on Virtual Execution Environments*. Association for Computing Machinery, New York, USA. DOI : <http://dx.doi.org/10.1145/1254810.1254834>
- [57] Ali J. Fahs and Guillaume Pierre. 2019. Proximity-aware traffic routing in distributed fog computing platforms. In *Proceedings of the 2019 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. 478–487. DOI : <http://dx.doi.org/10.1109/CCGRID.2019.00062>
- [58] Fikirte Tekla, Chung-Horng Lung, and Samuel Ajila. 2015. Seamless live virtual machine migration with cloudlets and multipath TCP. In *Proceedings of the 2015 IEEE 39th Annual Computer Software and Applications Conference*. 607–616. DOI : <http://dx.doi.org/10.1109/COMPSAC.2015.31>
- [59] Eric Harney, Sebastien Goasguen, Jim Martin, Mike Murphy, and Mike Westall. 2007. The efficacy of live virtual machine migrations over the Internet. In *Proceedings of the 2nd International Workshop on Virtualization Technology in Distributed Computing*. 1–7. DOI : <http://dx.doi.org/10.1145/1408654.1408662>
- [60] Laura Lemmi, Carlo Puliafito, Antonio Virdis, and Enzo Mingozzi. 2023. Ensuring lossless workload migration at the edge with SRv6. In *Proceedings of the 2023 IEEE Conference on Network Function Virtualization and Software Defined Networks*.
- [61] Carlo Puliafito, Antonio Virdis, and Enzo Mingozzi. 2020. The impact of container migration on Fog services as perceived by mobile things. In *Proceedings of the 2020 IEEE International Conference on Smart Computing*. 9–16. DOI : <http://dx.doi.org/10.1109/SMARTCOMP50058.2020.00022>
- [62] Andrej Binder, Tomas Boros, and Ivan Kotuliak. 2015. A SDN based method of TCP connection handover. In *Proceedings of the Information and Communication Technology*. Ismail Khalil, Erich Neuhold, A Min Tjoa, Li Da Xu, and Ilun You (Eds.), Springer International Publishing, Cham, 13–19.
- [63] Vitor A. Cunha, Daniel Corujo, Joao P. Barraca, and Rui L. Aguiar. 2020. Using Linux TCP connection repair for mid-session endpoint handover: A security enhancement use-case. In *Proceedings of the 2020 IEEE Conference on Network Function Virtualization and Software Defined Networks*. 174–180. DOI : <http://dx.doi.org/10.1109/NFV-SDN50289.2020.9289898>
- [64] Wissem Soussi, Maria Christopoulou, Themis Anagnostopoulos, Gürkan Gür, and Burkhard Stiller. 2023. TopoFuzzer – a network topology fuzzer for moving target defense in the telco cloud. In *Proceedings of the NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*. 1–5. DOI : <http://dx.doi.org/10.1109/NOMS56928.2023.10154367>
- [65] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: Programming protocol-independent packet processors. *SIGCOMM Computer Communication Review* 44, 3 (2014), 87–95. DOI : <http://dx.doi.org/10.1145/2656877.2656890>
- [66] C. Filsfils, P. Camarillo, J. Leddy, D. Voyer, S. Matsushima, and Z. Li. 2021. RFC 8986: Segment Routing over IPv6 (SRv6) Network Programming. RFC Editor, USA.
- [67] Daniel Fireman, Paulo Silva, Thiago Emmanuel Pereira, Luis Mafra, and Dalton Valadares. 2024. Prebaking runtime environments to improve the FaaS cold start latency. *Future Generation Computer Systems* 155 (2024), 287–299. <https://www.sciencedirect.com/journal/future-generation-computer-systems/vol/155/suppl/C>
- [68] Sumer Kohli, Shreyas Kharbanda, Rodrigo Bruno, Joao Carreira, and Pedro Fonseca. 2024. Pronghorn: Effective checkpoint orchestration for serverless hot-starts. In *Proceedings of the 19th European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA. DOI : <http://dx.doi.org/10.1145/3627703.3629556>
- [69] Lixiang Ao, George Porter, and Geoffrey M. Voelker. 2022. FaaSnap: FaaS made fast using snapshot-based VMs. In *Proceedings of the 17th European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 730–746. DOI : <http://dx.doi.org/10.1145/3492321.3524270>
- [70] Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. 2021. Benchmarking, analysis, and optimization of serverless function snapshots. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. New York, USA. DOI : <http://dx.doi.org/10.1145/3445814.3446714>
- [71] Maksym Planeta, Jan Bierbaum, Leo Sahaya Daphne Antony, Torsten Hoefer, and Hermann Härtig. 2021. MigrOS: Transparent live-migration support for containerised RDMA applications. In *Proceedings of the 2021 USENIX Annual Technical Conference*. USENIX Association, 47–63. Retrieved from <https://www.usenix.org/conference/atc21/presentation/planeta>

- [72] Carlo Puliafito, Luca Conforti, Antonio Virdis, and Enzo Mingozzi. 2022. Server-side QUIC connection migration to support microservice deployment at the edge. *Pervasive and Mobile Computing* 83 (2022), 101580. <https://www.sciencedirect.com/journal/pervasive-and-mobile-computing/vol/83/suppl/C>
- [73] Shunmugapriya Ramanathan, Abhishek Bhattacharyya, Koteswararao Kondepudi, and Andrea Fumagalli. 2024. Enabling containerized Central Unit live migration in 5G radio access network: An experimental study. *Journal of Network and Computer Applications* 221 (2024), 103767. <https://www.sciencedirect.com/journal/journal-of-network-and-computer-applications/vol/221/suppl/C>
- [74] Niklas Eiling, Jonas Baude, Stefan Lankes, and Antonello Monti. 2022. Cricket: A virtualization layer for distributed execution of CUDA applications with checkpoint/restart support. *Concurrency and Computation: Practice and Experience* 34, 14 (2022), e6474. DOI: <http://dx.doi.org/10.1002/CPE.6474>
- [75] Hongliang Liang, Qiong Zhang, Mingyu Li, and Jianqiang Li. Toward migration of SGX-enabled containers. In *Proceedings of the 2019 IEEE Symposium on Computers and Communications*.
- [76] Jinyu Gu, Zhichao Hua, Yubin Xia, Haibo Chen, Binyu Zang, Haibing Guan, and Jinming Li. 2017. Secure live migration of SGX enclaves on untrusted cloud. In *Proceedings of the 2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. 225–236. DOI: <http://dx.doi.org/10.1109/DSN.2017.37>
- [77] Fritz Alder, Arseny Kurnikov, Andrew Paverd, and N. Asokan. 2018. Migrating SGX enclaves with persistent state. In *Proceedings of the 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. 195–206. DOI: <http://dx.doi.org/10.1109/DSN.2018.00031>
- [78] Garrett McGrath and Paul R. Brenner. 2017. Serverless computing: Design, implementation, and performance. In *Proceedings of the 2017 IEEE 37th International Conference on Distributed Computing Systems Workshops*. IEEE, 405–410.
- [79] Open Container Initiative - OCI. Release Notices. Retrieved August 07, 2024 from <https://opencontainers.org/release-notices/overview/>. ([n. d.])
- [80] Xavier Castillo and Daniel P. Siewiorek. 1980. Performance-reliability model for computing systems. *NTIS, SPRINGFIELD, VA, 1980, 26* (1980). <https://apps.dtic.mil/sti/tr/pdf/ADA099482.pdf>
- [81] Andrzej Duda. 1983. The effects of checkpointing on program execution time. *Information Processing Letters* 16, 5 (1983), 221–229.
- [82] Chenghao Rong, Jessie Hui Wang, Jilong Wang, Yipeng Zhou, and Jun Zhang. 2024. Live migration of video analytics applications in edge computing. *IEEE Transactions on Mobile Computing* 23, 3 (2024), 2078–2092. DOI: <http://dx.doi.org/10.1109/TMC.2023.3246539>
- [83] Vladimir Yussupov, Uwe Breitenbücher, Frank Leymann, and Christian Müller. 2019. Facing the unplanned migration of serverless applications: A study on portability problems, solutions, and dead ends. In *Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing*.
- [84] Paul Lipton, Derek Palma, Matt Rutkowski, and Damian Andrew Tamburri. 2018. TOSCA solves big problems in the cloud and beyond! *IEEE Cloud Computing* 5, 2 (2018), 37–47.
- [85] Benjamin Michalowicz, Kaushik Kandadi Suresh, Bharath Ramesh, Aamir Shafi, Hari Subramoni, Mustafa Abduljabbar, and Dhabaleswar Panda. In-depth evaluation of a lower-level direct-verbs API on InfiniBand-based clusters: Early experiences. In *Proceedings of the 2023 IEEE International Parallel and Distributed Processing Symposium Workshops*. DOI: <http://dx.doi.org/10.1109/IPDPSW59300.2023.00065>
- [86] Lee Dryburgh and Jeff Hewett. 2005. *Signaling System No. 7 (SS7/C7): Protocol, Architecture, and Services*. Cisco press.
- [87] Intel Corporation. 2024. *Intel Trust Domain Extension (TDX) Module*. Technical Report. Intel.
- [88] Intel Corporation. 2024. *Intel® TDX Module Architecture Specification: TD Migration*. Technical Report. Intel.
- [89] Intel. Github: MigTD. Retrieved April 14, 2024 from <https://github.com/intel/MigTD>. ([n. d.])
- [90] AMD. *AMD SEV Secure Nested Paging Firmware ABI Specification*. Technical Report 56860. AMD.
- [91] Velero. 2023. Velero [Official Site]. (August 2023). Retrieved August 19, 2023 from <https://velero.io/>
- [92] OpenVZ. 2023. CRIU Image Streamer. Retrieved February 28, 2024 from <https://github.com/checkpoint-restore/criu-image-streamer>. (August 2023). .
- [93] KubeVirt. KubeVirt [Official Site]. Retrieved August 19, 2023 from <https://kubevirt.io/>. ([n. d.])
- [94] Tarik Taleb, Konstantinos Samdanis, Badr Mada, Hannu Flinck, Sunny Dutta, and Dario Sabella. 2017. On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration. *IEEE Communications Surveys and Tutorials* 19, 3 (2017), 1657–1681. DOI: <http://dx.doi.org/10.1109/COMST.2017.2705720>
- [95] Kiranpreet Kaur, Fabrice Guillemin, and Francoise Sailhan. 2023. Live migration of containerized microservices between remote Kubernetes Clusters. In *Proceedings of the IEEE Conference on Computer Communications Workshops*. 1–6. DOI: <http://dx.doi.org/10.1109/INFOCOMWKSHSP57453.2023.10225858>
- [96] Eryk Schiller, Jesutofunmi Ajayi, Silas Weber, Torsten Braun, and Burkhard Stiller. 2022. Toward a live BBU container migration in wireless networks. *IEEE Open Journal of the Communications Society* 3 (2022), 301–321. <https://ieeexplore.ieee.org/document/9709593>

- [97] Andrew Machen, Shiqiang Wang, Kin K. Leung, Bong Jun Ko, and Theodoros Salonidis. 2018. Live service migration in mobile edge clouds. *IEEE Wireless Communications* 25, 1 (2018), 140–147.
- [98] Luiz Fernando Bittencourt, Márcio Moraes Lopes, Ioan Petri, and Omer F. Rana. 2015. Towards virtual machine migration in Fog computing. In *Proceedings of the 2015 10th International Conference on P2P, Parallel, Grid, Cloud and Internet Computing*. 1–8. DOI : <http://dx.doi.org/10.1109/3PGCIC.2015.85>
- [99] Carlo Puliafito, Diogo M. Gonçalves, Márcio M. Lopes, Leonardo L. Martins, Edmundo Madeira, Enzo Mingozzi, Omer Rana, and Luiz F. Bittencourt. 2020. MobFogSim: Simulation of mobility and migration for fog computing. *Simulation Modelling Practice and Theory* 101 (2020), 102062. <https://www.sciencedirect.com/science/article/pii/S1569190X19301935>
- [100] Zhiqing Tang, Xiaojie Zhou, Fuming Zhang, Weijia Jia, and Wei Zhao. 2019. Migration modeling and learning algorithms for containers in Fog computing. *IEEE Transactions on Services Computing* 12, 5 (2019), 712–725. DOI : <http://dx.doi.org/10.1109/TSC.2018.2827070>
- [101] Monika Gill and Dinesh Singh. 2020. ACO based container placement for CaaS in Fog computing. *Procedia Computer Science* 167 (2020), 760–768. DOI : <http://dx.doi.org/10.1016/j.procs.2020.03.406>. International Conference on Computational Intelligence and Data Science. <https://www.sciencedirect.com/science/article/pii/S1877050920308723>
- [102] Marwa Gamal, Rawya Rizk, Hani Mahdi, and Basem E. Elnaghi. 2019. Osmotic bio-inspired load balancing algorithm in cloud computing. *IEEE Access* 7 (2019), 42735–42744.
- [103] Balamurugan Balusamy, Jayashree Sridhar, Dhamodaran Divya, and P. Venkata Krishna. 2015. Bio-inspired algorithms for cloud computing: A review. *International Journal of Innovative Computing and Applications* 6, 3–4 (2015), 181–202.
- [104] Michalis Mavrovouniotis, Changhe Li, and Shengxiang Yang. 2017. A survey of swarm intelligence for dynamic optimization: Algorithms and applications. *Swarm and Evolutionary Computation* 33 (2017), 1–17. <https://www.sciencedirect.com/science/article/pii/S2210650216302541>
- [105] Kumar Nishant, Pratik Sharma, Vishal Krishna, Chhavi Gupta, Kuwar Pratap Singh, Nitin, and Ravi Rastogi. 2012. Load balancing of nodes in cloud using ant colony optimization. In *Proceedings of the 2012 UKSim 14th International Conference on Computer Modelling and Simulation*. 3–8. DOI : <http://dx.doi.org/10.1109/UKSim.2012.11>
- [106] El Houssine Bourhim, Halima Elbiaze, and Mouhamad Dieye. 2019. Inter-container communication aware container placement in Fog computing. In *Proceedings of the 2019 15th International Conference on Network and Service Management*. 1–6. DOI : <http://dx.doi.org/10.23919/CNSM46954.2019.9012671>
- [107] Harshit Gupta, Amir Vahid Dastjerdi, Soumya K. Ghosh, and Rajkumar Buyya. 2017. iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge and Fog computing environments. *Software: Practice and Experience* 47, 9 (2017), 1275–1296.
- [108] Chen Xianqin, Wan Han, Wang Sumei, and Long Xiang. 2009. Seamless virtual machine live migration on network security enhanced hypervisor. In *Proceedings of the 2009 2nd IEEE International Conference on Broadband Network and Multimedia Technology*. 847–853. DOI : <http://dx.doi.org/10.1109/ICBNMT.2009.5347800>
- [109] Anupam Tamrakar. 2014. Security in live migration of virtual machine with automated load balancing. *International Journal of Engineering Research and Technology* 3, 12 (2014), 806–811.
- [110] Zeynep Mavus and Pelin Angin. 2019. A secure model for efficient live migration of containers. *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications* 10, 3 (2019), 21–44. <https://hdl.handle.net/11511/38239>
- [111] Boris Danev, Ramya Jayaram Masti, Ghassan O. Karame, and Srdjan Capkun. 2011. Enabling secure VM-vTPM migration in private clouds. In *Proceedings of the 27th Annual Computer Security Applications Conference*. Association for Computing Machinery, New York, NY, USA, 187–196. DOI : <http://dx.doi.org/10.1145/2076732.2076759>
- [112] Peiru Fan, Bo Zhao, Yuan Shi, Zhihong Chen, and Mingtao Ni. 2015. An improved vTPM-VM live migration protocol. *Wuhan University Journal of Natural Sciences* 20, 6 (2015), 512–520.
- [113] M. R. Watson, N. Shirazi, A. K. Marnerides, A. Mauthe, and D. Hutchison. 2016. Malware detection in cloud computing infrastructures. *IEEE Transactions on Dependable and Secure Computing* 13, 02 (2016), 192–205. DOI : <http://dx.doi.org/10.1109/TDSC.2015.2457918>
- [114] Mohamed Azab, Bassem M. Mokhtar, Amr S. Abed, and Mohamed Eltoweissy. 2016. Smart moving target defense for linux container resiliency. In *Proceedings of the 2016 IEEE 2nd International Conference on Collaboration and Internet Computing*. 122–130. DOI : <http://dx.doi.org/10.1109/CIC.2016.028>
- [115] Huangxin Wang, Fei Li, and Songqing Chen. 2016. Towards cost-effective moving target defense against DDoS and covert channel attacks. In *Proceedings of the 2016 ACM Workshop on Moving Target Defense*. Association for Computing Machinery, New York, NY, USA, 15–25. DOI : <http://dx.doi.org/10.1145/2995272.2995281>
- [116] Wissem Soussi, Maria Christopoulou, Gürkan Gür, and Burkhard Stiller. 2023. MERLINS – moving target defense enhanced with Deep-RL for NFV in-depth security. In *Proceedings of the 2023 IEEE Conference on Network Function Virtualization and Software Defined Networks*. 65–71. DOI : <http://dx.doi.org/10.1109/NFV-SDN59219.2023.10329594>

- [117] OpenVZ. Advanced Usage - CRIU. Retrieved August 07, 2024 from https://criu.org/Advanced_usage. ([n. d.]).
- [118] PaX Team. *Address Space Layout Randomization*. PaX Team. Retrieved 21 November 2024 from <https://pax.grsecurity.net/docs/aslr.txt>
- [119] Mojdeh Karbalaee Motalleb, Chafika Benzaïd, Tarik Taleb, and Vahid Shah-Mansouri. 2023. Moving target defense based secured network slicing system in the O-RAN architecture. In *Proceedings of the 2023 IEEE Global Communications Conference: Mobile and Wireless Networks*.
- [120] Sailik Sengupta, Tathagata Chakraborti, and Subbarao Kambhampati. 2019. MTDeep: Boosting the security of deep neural nets against adversarial attacks with moving target defense. In *Proceedings of the Decision and Game Theory for Security*. Springer International Publishing.
- [121] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 1877–1901. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [122] Anakin.ai. Can You Really Run Llama 3.1 405B Locally? Retrieved August 23, 2024 from <https://anakin.ai/blog/can-you-really-run-llama-3-1-405b-locally/>. ([n. d.]).

Received 26 April 2024; revised 29 October 2024; accepted 4 November 2024